
Synthetic Financial Time Series

Benchmarking generative and econometric models
on BRICS equity indices

Progress report

Victor Sobottka

Barcelona School of Economics (BSE)
in collaboration with Universitat Politècnica de Catalunya (UPC)

Scope. Five generators of daily equity returns — three generative adversarial networks and two GARCH baselines — trained per market on five BRICS indices, 2006–2026, and ranked with a metric suite audited against a shuffled copy of the real data.

2026-09-18

Contents

Provenance

Every number in this report is read from the pipeline run recorded below by `report_data.py`;

Generated	2026-09-12T13:18:16 (git a53060cc5c)
Seeds	42, 43, 44
Markets	BOVESPA, FTSE, MOEX, NIFTY50, SHANGHAI
Walk-forward folds	5
Generator updates	9,000
Software	Python 3.12.3 · PyTorch 2.13.0+cu130
Libraries	arch 8.0.0 · statsmodels 0.15.0 · scipy 1.18.1 · numpy 2.5.3 · pandas 2.3.3
CUDA	yes (NVIDIA GeForce RTX 4090)
Platform	Linux-6.8.0-106-generic-x86_64-with-glibc2.39

The numerical library versions are recorded because they are load-bearing, not as bookkeeping. Whether a GARCH fit that reaches the integrated boundary generates an explosive path flipped between the production environment of an earlier run and a local refit of the same data window (Section ??); a difference in the arch version is the likeliest explanation, and that earlier run did not record it.

1 Highlights

- **A 6-parameter model ranks first.** GJR-GARCH, fitted in 0.018 s, has the best mean composite rank (2.524). The largest generator, QuantGAN, has 276,481 parameters and trains for 481 s per market–seed, and ranks below it on composite, on fidelity and on temporal.
- **The margin is thin, and the two families win different things.** CNN-WGAN-GP trails by 0.081 on composite. The gradient-trained family takes 10 of 15 per-evaluation composite wins and 9 of 15 fidelity wins; the econometric family takes 9 of 15 temporal wins.
- **A shuffled copy of the real data beats most generators on volatility clustering.** 9 of 19 computed metrics cannot tell a shuffled deck from the real series at all. On the autocorrelation of squared returns the shuffled control beats all 3 GANs, and only GJR-GARCH clears it.
- **GARCH fits reach the integrated boundary as a property of the data.** 12 of 180 econometric fits converge to persistence 1, in 4 market–window–model combinations and identically in every seed. A refit under a stationarity bound costs 0.000581 to 0.019743 in log-likelihood.
- **Most generators remain distinguishable from real data.** In walk-forward evaluation 3 of 5 models sit more than 1.96 null standard deviations from chance on discriminative AUC. AUC falls from 0.811 to 0.664 as the training window grows, and the decline is not monotonic.
- **Downstream, the pooled risk backtest favours CNN-WGAN-GP.** Across 3 backtests over $n = 2,479$ pooled observations it has the lowest median QLIKE (−8.134).
- **Mode collapse is caught by instrumentation, not by eye.** The post-generation guard fired 18 times across 2 models (TimeGAN (16), QuantGAN (2)); TimeGAN is the worst of the five on all 7 temporal metrics.

2 Why this work matters

Synthetic market data is wanted where real data cannot serve: crisis episodes are too rare to train or stress-test risk models on, proprietary positions cannot be shared, and scenario analysis needs many plausible histories rather than the single one that happened. Generative adversarial networks have been proposed for this, and the reference studies — TimeGAN^[4] and QuantGAN^[5] — report that they reproduce the statistical regularities of returns, the *stylised facts*^[7], better than parametric baselines.

Evaluation is the hard part. A generator can reproduce the distribution of daily returns exactly and still be useless for risk work if it scatters large moves at random, because what a risk model needs is the way turbulence clusters in time. Many of the metrics used to judge synthetic series look only at the distribution. This study measures that directly: scored by the same code as the generators, a random permutation of the real series — identical values, no dynamics — is indistinguishable from the real data on 9 of the 19 metrics computed here. A benchmark that does not audit its metrics against such a control cannot tell a working generator from a shuffled deck.

Emerging markets make the test harder in the ways that matter. The reference GAN studies evaluate on developed-market data, for example the S&P 500 in Wiese et al.^[5]. The five BRICS indices used here span the 2008 crisis, the 2015 Chinese market crash, the COVID shock and the suspension of the Moscow exchange in 2022, and they carry heavy tails and strong volatility clustering: across 24,758 market-days, 76 daily moves exceed five standard deviations, where a Gaussian model predicts fewer than one (Table ??). A generator that works on these series is doing something a developed-market benchmark would not reveal.

The study adds a second test that most benchmarks omit: two econometric models, GARCH(1,1)^[39] and GJR-GARCH(1,1)^[40], fitted on the same data and scored by the same pipeline. A benchmark of generators against each other always produces a winner. Including the parametric baseline as a genuine competitor is what shows whether the winner is useful.

3 Setting and design

3.1 Data

The data are daily closing prices of five BRICS equity indices — BOVESPA (Brazil), FTSE/JSE All Share (South Africa), MOEX (Russia), NIFTY 50 (India) and the Shanghai Composite (China) — from 2006-08-30 to 2026-08-28, twenty years and 24,758 market-days of log returns in total. Each market is split in time into 80% training, 10% validation and 10% test, never shuffled. Table ?? summarises the returns.

Across 24,758 market-days a Gaussian model predicts fewer than 0.01 days with $|r| > 5\sigma$; the data contain 76, with MOEX alone contributing 21. Large moves also cluster: in MOEX the probability of a move beyond 2σ is 3.8%, and 22.9% on the day after one, a ratio of 6.1; the ratio lies between 3.4 and 6.1 in all five markets.

Table 1. The five return series, computed from the processed daily log returns. σ : daily standard deviation. Excess kurtosis is zero for a Gaussian. Residual kurtosis enters the pipeline only as a real-versus-synthetic difference, so it has no standalone value per market.

Market	σ (%/day)	Excess kurtosis	Residual kurtosis
BOVESPA	1.63	10.36	—
FTSE	1.20	5.72	—
MOEX	1.87	65.63	—
NIFTY50	1.29	14.52	—
SHANGHAI	1.48	5.77	—

Why MOEX, not MSCI World — and what it costs. An earlier version of this basket used MSCI World as the Russia leg. MSCI World is a developed-market global index and is not a BRICS market; using it made the “BRICS emerging market” framing false. MOEX replaces it. The cost is a genuine structural break: a -33.3% single-day return on 2022-02-24 followed by a 27-trading-day suspension (2022-02-25 to 2022-03-24). Both are recorded as a known gap and never interpolated — interpolating would manufacture returns on days when no trading occurred, in exactly the regime the benchmark is meant to stress. MOEX consequently dominates several descriptive statistics, which is a property of the data, not an artefact.

Log returns, and no clipping. $r_t = \ln(P_t/P_{t-1})$, dates parsed with an explicit format and sorted chronologically before any other operation (lexicographic sorting of MM/DD/YYYY is wrong across year boundaries). No outlier clipping. Clipping at the 0.5th/99.5th percentiles would remove precisely the observations that determine kurtosis, tail index and the extreme-events metric — the three properties motivating the BRICS choice — and Adams et al. (2019)^[6] show winsorising worsens distributional misfit.

Splits, folds and windows. Temporal 80/10/10, never shuffled. 128-step sliding windows, stride 1. Walk-forward validation uses 5 rolling folds; each fold retraines from scratch on that fold’s training segment.

3.2 Two metric families, and why the split exists

Every metric compares a synthetic series with the real test series, and every metric belongs to one of three groups. *Fidelity* metrics depend only on the marginal distribution of returns, so they are unchanged if the series is shuffled. *Temporal* metrics depend on the order of the observations: autocorrelation, long memory, conditional tails and a classifier that sees windows of consecutive days. *Descriptive* metrics are computed and reported but never ranked, each for a reason given below. Seven fidelity and seven temporal metrics are ranked.

Why the ranking splits fidelity from temporal. *Evidence:* an unweighted mean over all metrics was won by the shuffled-real control, avg_rank 1.24 against 2.47 for a genuine generator. 9 of the 19 computed metrics are permutation-invariant, so they score a shuffled deck perfectly by construction, and no weighting of the remainder can overcome that. The composite is therefore

$$\text{composite_rank} = \frac{1}{2} (\text{fidelity_rank} + \text{temporal_rank}),$$

the equal-weighted mean of the two families — families are weighted, not individual metrics. Model selection uses composite_rank; avg_rank is retained for comparability only and is never the selection criterion.

Why the control does not compete for rank. Its fidelity_rank would be 1.000 by construction — it *is* the real data, reordered — so no weighting scheme can prevent it winning that family. It is scored on every metric, its raw values are reported in full, and its rank columns are NaN. Any generator scoring worse than the control on temporal metrics has learned nothing about dynamics, which makes it a diagnostic rather than a competitor.

Why NaN ranks last. Ranking uses na_option='bottom', so a metric that fails to compute counts as the worst outcome rather than being silently dropped. Dropping it would reward a model for producing output degenerate enough to break an estimator.

3.3 The metrics

Table ?? lists every quantity the pipeline computes. r_t is the real daily log return and $\tilde{r}_t$ the synthetic one; a tilde marks the synthetic counterpart of any statistic; n is the test-series length. Unless stated, “difference” means the absolute difference between the real and synthetic value, and lower is better.

Table 2. Every metric the pipeline computes: what it measures, how it is computed, where it comes from, and whether it is ranked. “This work” marks a metric defined for this study rather than taken from the literature.

Metric	Measures	Computation	Source	Role
<i>Fidelity: the marginal distribution (unchanged by shuffling)</i>				
mean_diff	Location	$ \bar{r} - \tilde{\bar{r}} $	standard moment	ranked
std_diff	Scale	difference of sample standard deviations	standard moment	ranked
wasserstein	Distance between distributions	one-dimensional Wasserstein-1 distance between the two empirical distributions	Kantorovich–Rubinstein distance; GAN objective in ^[2]	ranked
energy_distance	Distance between distributions	$2\mathbb{E} X - Y - \mathbb{E} X - X' - \mathbb{E} Y - Y' $ on random subsamples of at most 500 points each (seed 42)	Székely & Rizzo ^[11]	ranked
quantile_mse	Fit of body and tails	mean squared difference of the 1, 5, 10, 25, 50, 75, 90, 95 and 99% quantiles	this work	ranked
tail_index_diff	Tail heaviness	difference of Hill tail-index estimates from the largest 5% of $ r_t $; an estimate above 20 counts as a failure	Hill ^[38]	ranked
extreme_events_diff	Frequency of large moves	difference in the percentage of days with $ r_t > 2\sigma$, each series against its own σ	stylised fact ^[7] ; metric this work	ranked
<i>Temporal: dynamics (changed by shuffling)</i>				

Table ??, continued

Metric	Measures	Computation	Source	Role
acf_returns_mae	Linear predictability	mean absolute difference of sample autocorrelations of r_t over lags $1, \dots, L$, with $L = \min(50, \lfloor n/4 \rfloor)$	Cont ^[7]	ranked
acf_absolute_mae	Volatility clustering	as above, for $ r_t $	Ding, Granger & Engle ^[8]	ranked
acf_squared_mae	Volatility clustering (ARCH effects)	as above, for r_t^2	Engle ^[10] ; Cont ^[7]	ranked
hurst_diff	Long memory in volatility	difference of rescaled-range (R/S) Hurst exponents of $ r_t $	Hurst ^[23] ; Mandelbrot & Wallis ^[24]	ranked
resid_kurtosis_diff	Tails beyond GARCH	difference in excess kurtosis of standardised residuals of a GARCH(1,1) fitted by Gaussian quasi-maximum likelihood to $100 r_t$	Bollerslev ^[36]	ranked
discriminative_auc_dist	Indistinguishability	$ \text{AUC} - 0.5 $ for a logistic regression separating real from synthetic 20-day windows described by mean, standard deviation, mean $ r_t $ and mean r_t^2 ; standardised features, five-fold unshuffled cross-validation	discriminative score of Yoon et al. ^[4] ; CTBench ^[16]	ranked
discriminative_auc_absz	Indistinguishability against a null	$ \text{AUC} - \mu_0 /\sigma_0$, with μ_0 and σ_0 from 20 real-versus-real half-splits of the same test series	this work	ranked
<i>Descriptive: computed per series and reported, never ranked</i>				
skewness_diff	Asymmetry	difference of sample skewness	standard moment	descriptive
kurtosis_diff	Tail weight	difference of sample excess kurtosis	standard moment	descriptive
arch_pvalue_diff	ARCH effects	difference of Engle ARCH-LM p -values, 5 lags	Engle ^[10]	descriptive
arch_stat_diff	ARCH effects	difference of ARCH-LM statistics, 5 lags	Engle ^[10]	descriptive
discriminative_auc_raw	Classifier AUC	the AUC above, unsigned	Yoon et al. ^[4]	descriptive
<i>Downstream utility: one pooled backtest per seed, reported beside the ranking</i>				
qlike	Variance-forecast quality	$\frac{1}{n} \sum_t (\log \sigma_t^2 + r_t^2/\sigma_t^2)$ on real returns, with σ_t^2 filtered from GARCH(1,1) parameters fitted to the synthetic series	Patton ^[37]	reported
kupiec_p	VaR coverage	likelihood-ratio test that violations occur at the nominal 5% rate	Kupiec ^[34]	reported
christoffersen_p	VaR independence	likelihood-ratio test that violations do not cluster	Christoffersen ^[35]	reported
var_coverage_error	VaR violation rate	$ v/n - 0.05 $ for v violations of a 5% conditional value-at-risk in the pooled backtest	Kupiec ^[34]	descriptive
garch_persistence_diff	Volatility persistence	difference of $\alpha + \beta$ between Gaussian GARCH(1,1) fits to the pooled synthetic and pooled real series	this work	descriptive

Within each market–seed evaluation the five generators are ranked on each ranked metric. The fidelity and temporal ranks are the mean rank within each family, and the composite rank is the mean of the two.

The Kolmogorov–Smirnov statistic is drawn in the figures for orientation but is not a metric, because its p -values assume independent observations that daily returns are not (Appendix ??).

3.4 Design decisions and their evidence

Each choice below is stated with the measurement that motivated it. Where that measurement comes from an earlier run or a separate experiment rather than from this run’s artifacts, it is a prior measurement; the numbers are recorded in the repository’s standing documentation.

Why walk-forward, not random CV. Shuffled cross-validation leaks. Our 20-day evaluation windows overlap by 19 observations, so `shuffle=True` places near-duplicate windows in both train and test. Measured, this moves the discriminative-AUC null from 0.506 to 0.584 — a spurious improvement of the same order as the differences between models (Tashman 2000^[13]; Bergmeir & Benítez 2012^[14]).

Why per-market training, not cross-market pooling. Each model trains on one market’s own data; markets are never concatenated. Pooling was considered: joining all five end-to-end introduces only ≈ 4 spurious cross-market transitions across $\approx 3,900$ windows, a small artefact in exchange for more training volume. It was rejected because a single generator fitted across five independent markets risks each market’s dynamics contaminating the others’ learned distribution — and the object of study is per-market stylized facts.

Why budgets are in gradient steps, never epochs. An epoch is $\lfloor n_{\text{windows}}/\text{batch_size} \rfloor$ steps — a data-dependent unit disguised as a fixed one. Walk-forward folds differ in length by a factor of five (Table ??), so a fixed epoch count silently gives later folds five times the training. *Evidence:* specifying budgets in epochs once produced a $492\times$ asymmetry in generator updates between models in a single run, because a smoke-test override of epochs reached two models and not the third. The affected model looked like an architectural failure until the budget was actually measured. All budgets, seeds, markets and fold counts are therefore declared in one configuration cell and nowhere else.

Why parity is on generator updates, and scoped to the gradient family. The pipeline asserts `TimeGAN.joint_steps == QuantGAN.train_steps == CNN-WGAN-GP.train_steps` and refuses to run otherwise. TimeGAN’s `ae_steps` and `sup_steps` are pre-training required by its four-phase algorithm (Yoon et al. 2019^[4]); they are reported separately and excluded from parity, because counting them would penalise the algorithm for its own structure. QuantGAN and CNN-WGAN-GP take $n_{\text{critic}} = 5$ discriminator updates per generator update (Gulrajani et al. 2017^[3]), intrinsic to WGAN-GP rather than extra budget. The econometric models are exempt entirely: maximum-likelihood fitting has no gradient-step analogue, and any number entered in that column would be fabricated.

Equal generator updates is not equal compute, and both are reported because they are different claims. Measured at 1,000 generator steps over five runs: TimeGAN ≈ 20.5 s, CNN-WGAN-GP ≈ 42.4 s, QuantGAN ≈ 178 s — a $\approx 9\times$ spread at identical budget. Full-run wall-clock is in Results, Table ??.

Why z-score + $\tanh(z/3)$, not min-max. All three GANs normalise with $\tanh(z/3)$ on the z-score, mapping to $[-1, +1]$ and inverting via $z = 3 \operatorname{arctanh}(y)$. Min-max was rejected on measurement. At tail indices of 2.54–3.21 a single extreme day sets the whole scale: min-maxing MOEX to $[-1, 1]$ — its extremes are -40.5% and $+25.2\%$ — leaves 98% of the data occupying 14.3% of the range with a median at +0.233, so the generator must learn an offset before it can learn any dynamics. *Evidence:* changing only this moved QuantGAN’s output standard deviation from $3.7\times$ real to $1.2\times$ real at identical budget. Sigmoid to $[0, 1]$ is doubly wrong here — it makes negative returns unrepresentable and $\operatorname{arctanh}$ explodes as $y \rightarrow 1$.

Why GARCH is fitted on raw returns instead. The econometric models are fitted on raw log returns scaled by 100, *not* the project normalisation. The tanh squash compresses exactly the variance dynamics GARCH exists to model, so applying it would handicap the baseline on its own ground. The $\times 100$ scaling is numerical: arch’s optimiser converges poorly at magnitudes near 10^{-2} .

Why two GARCH variants and not more. $\text{GARCH}(1,1)^{[39]}$ is the universal baseline and the natural null. GJR-GARCH^[40] adds one leverage term, capturing Cont’s stylized fact 5 (gain/loss asymmetry)^[7] — which none of the GANs models explicitly, and which is the single most-cited deficiency of symmetric GARCH on equity data. Both use Student- t innovations. Stopping at two keeps the baseline serious without turning a GAN benchmark into a GARCH paper; the composite ranking places GJR-GARCH 1st and GARCH 3rd of 5, which is consistent with the leverage term earning its place.

Why kurtosis_diff and skewness_diff are descriptive-only. Hill $\hat{\alpha}$ is 2.54–3.21 across the five markets, so $\mathbb{E}|X|^k < \infty$ only for $k < \alpha$: variance exists everywhere, skewness is infinite in four of five markets, and **kurtosis is infinite in all five**. A sample statistic estimating a population quantity that does not exist has nothing to converge to — it grows with the window instead.

Evidence, mean excess kurtosis over disjoint blocks of increasing length (disjoint, so each estimate is independent rather than reusing the shorter windows’ data):

Market	$n = 250$	$n = 500$	$n = 1,000$	$n = 2,000$
BOVESPA	2.00	3.64	7.02	10.09
FTSE	1.77	2.49	3.74	6.35
MOEX	7.58	20.11	46.14	107.70
NIFTY50	2.95	5.48	7.48	14.61
SHANGHAI	4.28	3.96	3.91	5.40

The estimate grows with n in 5 of 5 markets, by factors of $1.3\times$ to $14.2\times$ between blocks of 250 and 2,000 observations. Both metrics are computed and displayed; neither is ranked. `tail_index_diff`, the Hill estimator^[38] on the top 5% of $|r|$, is the ranked heavy-tail metric — it estimates α itself, which does exist. Hill returns NaN above $\hat{\alpha} = 20$, after a degenerate series once drove it to 31,581.

Why both ARCH-LM variants are descriptive-only. `arch_pvalue_diff` identifies the better-fitting model 99% of the time on simulated GARCH but saturates completely on real data: real and shuffled both underflow to $p = 0.0$, giving $|\Delta p| = 0.000000$ and rating the adversarial control a perfect match. `arch_stat_diff` scores 76% with a null sd of 46.4 — two draws from the same process gave LM 56.8 and 124.8. Neither is reliable across both regimes. ACF-MAE replaces them as the primary volatility-clustering signal: continuous, no saturation, 88% Monte Carlo accuracy.

Why discriminative AUC is ranked on $|\text{AUC} - 0.5|$. *Evidence:* under an ascending rank on the raw value, an anti-predictive AUC of 0.30 outranked an indistinguishable 0.50 — the optimum is 0.5, not 0. The empirical null is 0.506 ± 0.084 over 15 real-vs-real half-splits, not exactly 0.5, so a z -score against that null is reported as a second ranking metric and an observed 0.62 is not by itself evidence of failure ($z = 1.36$).

Why downstream utility uses conditional VaR. Unconditional VaR was tested first and added nothing: Gaussian iid noise scored identically to real data (coverage error 0.0276 for both), because an unconditional quantile probes only the marginal — already covered by Wasserstein and the quantile MSE. Conditional VaR probes whether the GARCH structure itself transfers, which is the question TSTR is asking.

Why QLIKE needs a large n , and what that means here. QLIKE inverts at small samples: at $n \approx 126$ Gaussian noise scored -6.888 against real data's -6.876 , ranking noise above real. At $n = 623$ pooled it ranks correctly with sd 0.000. The pipeline therefore computes it once per seed over every market's concatenated test set: n is 2,479 here (Table ??), which meets the $n \gtrsim 600$ threshold, at the disclosed cost of the variance filter crossing 4 market boundaries.

Why Gaussian QMLE for the residual-kurtosis metric. A Student- t specification absorbs excess kurtosis by construction and makes the test uninformative for every model. Gaussian QMLE keeps it able to fail. Circularity with the GARCH generators is discussed in Results.

Why determinism is not enforced. `torch.use_deterministic_algorithms(True)` and `cuda.deterministic` are deliberately unset; the cost is measured rather than assumed (Section ??). Consequence, stated as a rule: no single-run winner is reported on `tail_index_diff`, `hurst_diff` or `mean_diff`, because those three change their winner between identical repeat runs.

Instrumentation retained. The [DIAG] latent statistics, the [WARNING] post-generation guard and the step-count printout are kept in the pipeline permanently. Each fires rarely; between them they located latent collapse, mode collapse and the $492\times$ budget gap. The guard's firings in this run are a reported result (Section ??), not a debug artefact.

4 Architecture comparison

The benchmark compares five generators in two families. The three *gradient-trained* models are generative adversarial networks trained on windows of 128 consecutive returns to an asserted parity of 9,000 generator updates: TimeGAN^[4], a recurrent autoencoder with an adversarial latent model; QuantGAN^[5], a temporal convolutional network trained as a Wasserstein GAN with gradient penalty (WGAN-GP)^[3]; and CNN-WGAN-GP, a transposed-convolution generator trained with the same objective. The CNN-WGAN-GP is named for its architecture and is distinct from the published Fin-GAN^[41], which forecasts and classifies returns with an economics-driven loss; this one is an unconditional generator and makes no forecasts. The two *econometric* models, GARCH(1,1)^[39] and GJR-GARCH(1,1)^[40] with Student- t innovations, are fitted

by maximum likelihood on the market’s raw returns. Table ?? sets them side by side.

Table 3. The five generators. Budget parity binds the three gradient-trained models; maximum-likelihood fits have no gradient-step analogue and are exempt.

	TimeGAN	QuantGAN	CNN-WGAN-GP	GARCH(1,1)- t	GJR-GARCH- t
Family	gradient	gradient	gradient	econometric	econometric
Core	4-phase (embedder + supervisor + GAN), GRU backbone	TCN backbone, WGAN-GP	CNN deconvolution, WGAN-GP	Conditional variance, Student- t innovations	GARCH + leverage indicator
Temporal mechanism	Recurrent: each step in order, gating what to remember	Dilated causal convolutions: all time scales in one pass	Transposed convolutions: upsample noise to full sequence	$\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2$	adds $\gamma r_{t-1}^2 \mathbb{I}[r_{t-1} < 0]$
Fitting	BCE + moment matching, 4 phases	WGAN-GP, $n_{\text{critic}} = 5$, $\lambda_{\text{gp}} = 10$	WGAN-GP, $n_{\text{critic}} = 5$, $\lambda_{\text{gp}} = 10$	ML; constrained refit if persistence reaches 1	ML; constrained refit if persistence reaches 1
Reference	Yoon et al. 2019 ^[4]	Wiese et al. 2020 ^[5]	This work; not Fin-GAN ^[41]	Bollerslev 1986 ^[39]	Glosten et al. 1993 ^[40]
Key settings	hidden_dim 24, layers 3, lr 1e-3	noise_dim 100, lr 1e-4, 3 TCN blocks (dil. 1/2/4)	base_channels 64, lr 1e-4, 3×ConvTranspose1d	$p=1, q=1, o=0$, dist t , burn-in 500	$p=1, q=1, o=1$, dist t , burn-in 500
Input scaling	z-score + $\tanh(z/3) \rightarrow [-1, 1]$	z-score + $\tanh(z/3) \rightarrow [-1, 1]$	z-score + $\tanh(z/3) \rightarrow [-1, 1]$	Raw returns ×100	Raw returns ×100
seq_len constraint	any	any	divisible by 8	n/a	n/a
Budget	generator-update parity (asserted)	parity	parity	exempt — ML fit, no step analogue	exempt
Inductive bias	Step-by-step ordering; persistence via GRU state	Multi-scale memory; dilations span short and long horizons	Coarse-to-fine hierarchical refinement, borrowed from image GANs	Conditional heteroskedasticity, symmetric response	As GARCH, plus asymmetric response to negative returns (Cont fact 5 ^[7])

Table ?? records the training regime’s cost. Sampling cost — the time a fitted model takes to generate a series — was not recorded by this run, so it is not reported. Equal generator updates is not equal compute: QuantGAN and CNN-WGAN-GP each take five critic updates per generator update^[3], and TimeGAN’s 3,000 autoencoder and 3,000 supervisor pre-training steps are excluded from parity and reported separately^[4].

4.1 Settings shared by the two WGAN-GP models

$n_{\text{critic}} = 5$: the critic must estimate the Wasserstein distance accurately before the generator uses its gradient; fewer updates leave it under-fitted and the gradient biased. Gulrajani et al. (2017)^[3] establish this as a floor and CTBench^[16] and SFAG^[28] fix it identically.

$\lambda_{\text{gp}} = 10$: values below 5 permit Lipschitz violations, invalidating the Kantorovich–Rubinstein duality the training objective rests on. 10 is the standard from Gulrajani et al. and has not been improved on.

$\beta_1 = 0$ in **Adam**: in adversarial training the correct generator direction reverses each time the critic updates, so momentum accumulates stale direction and pushes the wrong way. $\beta_1 = 0$ uses the current

Table 4. Size and cost of each model against its composite rank (Table ??). *Params*: trainable generator parameters for the gradient-trained models, fitted parameters for the econometric ones (GARCH(1,1)- t : $\mu, \omega, \alpha, \beta, \nu$; GJR-GARCH adds γ). *Gen. updates*: n/a for maximum-likelihood fits, which have no gradient-step analogue, rather than an invented number. *Fit*: mean wall-clock seconds to train or fit one model, over every market-seed.

Model	Family	Params	Gen. updates	Fit (s)	Composite
GJR-GARCH	econometric	6	n/a	0.018	2.524
GARCH	econometric	5	n/a	0.023	2.967
TimeGAN	gradient	11,400	9,000	81.1	3.864
CNN-WGAN-GP	gradient	55,401	9,000	180.4	2.605
QuantGAN	gradient	276,481	9,000	481.1	3.041

gradient only.

LayerNorm, not BatchNorm, in the CNN-WGAN-GP critic: the gradient penalty needs the critic’s gradient norm at a *single* interpolated point. BatchNorm makes that value depend on the other samples in the batch, corrupting the penalty.

4.2 TimeGAN’s four-phase training

Phase 1 — autoencoder: train embedder $e : \mathcal{X} \rightarrow \mathcal{H}$ and recovery $\hat{r} : \mathcal{H} \rightarrow \mathcal{X}$ on $\|X - \hat{r}(e(X))\|^2$.

Phase 2 — supervisor: train $s : \mathcal{H} \rightarrow \hat{\mathcal{H}}$ on $\|H_{t+1} - s(H_t)\|^2$, injecting temporal causality into the latent space.

Phase 3 — joint adversarial: $z \rightarrow G(z) \rightarrow s(G(z)) \rightarrow \hat{r}(\cdot) \rightarrow \tilde{X}$, with

$$\begin{aligned}\mathcal{L}_G &= \mathcal{L}_U + 100\sqrt{\mathcal{L}_S} + 100 \mathcal{L}_V \\ \mathcal{L}_V &= |\mu_H - \mu_{\hat{H}}| + |\sigma_H - \sigma_{\hat{H}}|\end{aligned}$$

Phase 4 — fine-tuning of the recovery network on generated sequences.

Phases 1–2 are pre-training and are excluded from budget parity; only phase-3 joint steps are counted, matching QuantGAN’s and CNN-WGAN-GP’s `train_steps`. In this run the pre-training budget was 3,000 autoencoder and 3,000 supervisor steps, reported here rather than folded into the parity figure.

5 Pipeline

Figure ?? shows the pipeline from price files to ranked results. Each stage is described below in the order it runs.

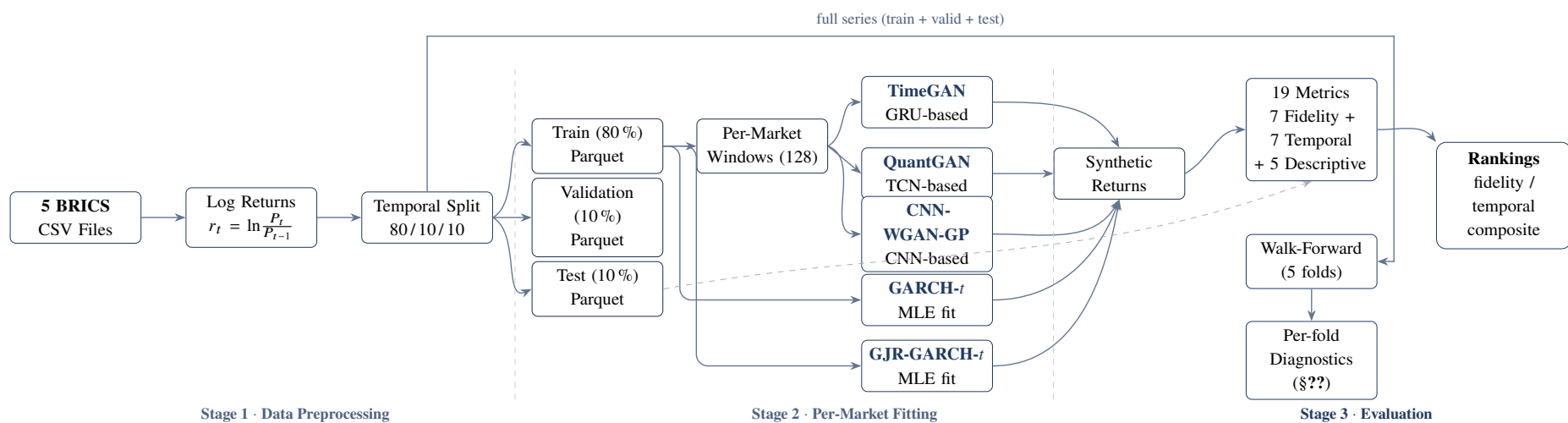


Figure 1. The pipeline. Data are prepared once; every model is then fitted independently per market and seed, evaluated against the real test series and in walk-forward folds, and ranked.

5.1 Reading the diagram

Figure ?? runs left to right in three stages, separated by the dashed vertical rules. **Stage 1** turns the five markets’ CSV files into log returns and splits each market’s series in time into Train (80 %), Validation (10 %) and Test (10 %) Parquet files; the training split is then cut into overlapping 128-step windows. **Stage 2** fits every model independently per market and seed: the three gradient-trained generators (TimeGAN, QuantGAN, CNN-WGAN-GP) read the windowed training data, while the two econometric models (GARCH- t , GJR-GARCH- t) are fed directly from the raw training returns, bypassing the windowing node — the $\tanh(z/3)$ squash used to bound the GAN inputs would compress exactly the variance dynamics GARCH exists to model (Section ??). All five write to Synthetic Returns.

Stage 3 is where the diagram forks into two tracks that both read from the Temporal Split node and never from each other.

Track A, along the bottom, is the fixed-split evaluation: Test and Synthetic Returns both feed 19 Metrics, which feeds Rankings. This is Table ?? — fidelity, temporal and composite rank on one held-out test series per market and seed, 15 evaluations in total, one training per model per evaluation: $5 \text{ models} \times 15 \text{ evaluations} = 75 \text{ trainings}$.

Track B, along the top, is walk-forward: an arrow leaves the Temporal Split node directly, labelled “full series (train + valid + test)”, and feeds Walk-Forward, which terminates in its own Per-fold Diagnostics node (Section ??) rather than in Rankings — composite rank never includes a walk-forward number. Walk-forward retrains every model from scratch on each of 5 rolling-origin folds, so it runs $5\times$ as many trainings as Track A: $5 \text{ folds} \times 5 \text{ models} \times 15 \text{ evaluations} = 375 \text{ trainings}$, against Track A’s 75. That is why walk-forward dominates the pipeline’s wall-clock time, independent of any per-model cost difference (Table ??).

Track B reassembles the complete series rather than reading the Test split because the test split alone is too short for what it evaluates: at 496 points, walk-forward’s rolling-origin folds ranged 168–414 points, which at `seq_len = 128` and `batch_size = 64` gives 0–2 batches per epoch — the gradient-trained models were evaluated essentially untrained, and discriminative AUC read 1.000 in every fold as a result (Section ??, Walk-forward validation). Reassembling the full series before folding is the fix, and it has a consequence worth stating before it is found some other way: Track A and Track B overlap in what they see. Walk-forward’s later folds train on data that includes the years Track A held out as its own test split. This is inherent to rolling-origin evaluation on a single series — there is no second independent series to hold out a second time (Tashman 2000^[13]; Bergmeir & Benítez 2012^[14]) — which is why the two tracks are reported separately throughout, Table ?? against Table ??, and never pooled into one score.

5.2 Data preparation

Closing prices are read with an explicit date format and sorted chronologically before anything else is computed; daily log returns are $r_t = \ln(P_t/P_{t-1})$, with no clipping of outliers. Each market’s series is split in time into training, validation and test parts (80/10/10) and stored as Parquet files. For the gradient-trained models the training part is cut into overlapping windows of 128 returns with stride 1.

5.3 Training with budget parity

Every model is trained or fitted independently for each market and each seed (42, 43, 44), on that market's training split only. The three GANs standardise returns and squash them with $\tanh(z/3)$, and train for exactly 9,000 generator updates; the pipeline asserts that the three budgets are equal and refuses to run otherwise. TimeGAN first runs its autoencoder and supervisor pre-training. The two econometric models are fitted by maximum likelihood on returns scaled by 100, in two stages: an unconstrained fit whose persistence is always recorded, and a refit under a stationarity bound only when that persistence reaches 1 (Section ??). Weights and fitted parameters are saved for every production model.

5.4 Walk-forward validation

Walk-forward validation re-runs training on growing windows of the full series. With n observations and $K = 5$ folds, each test block has $L = \lfloor n/(K + 1) \rfloor$ observations; fold k trains a fresh model on the first $n - L(K - k)$ observations and generates a series the length of the following block. Training data therefore always precede test data, and later folds train on more history. Each fold records Wasserstein and energy distance, quantile MSE, the Hurst and ARCH-LM differences and discriminative AUC.

5.5 Evaluation

Each fitted model generates one series with as many observations as the market's real test split. Every metric in Table ?? compares it with the real test series. A shuffled control — a fixed random permutation of the same real test series — is scored by the same code. For downstream utility each model's per-market draws are saved, then concatenated across all markets for each seed, and a single variance-forecasting backtest is run on the pooled sample.

5.6 Ranking

Within each market–seed evaluation, the five generators are ranked on every ranked metric; the control is excluded, and a metric that cannot be computed ranks last. Fidelity and temporal ranks are the means within each family and the composite rank is their mean. Ranks are then averaged over all 15 evaluations.

5.7 Reporting

Per-market results are written to disk as each evaluation completes, and aggregates are read back from disk rather than from memory, so an interrupted run resumes without losing completed work; a completed evaluation is skipped only if its recorded configuration — budgets, folds, model roster and code version — matches the run about to start. This report is built by `generate_report.py` from those files through `report_data.py`, which raises rather than substituting a value when an artifact is missing. A regression check, `verify_notebook.py`, guards the pipeline's safeguards before every commit.

6 Results

All tables pool 15 evaluations: 5 markets $\times$ 3 seeds. The figures show one of them, BOVESPA with seed 42, as an illustration of what the metrics measure; the rankings rest on the tables, not on the figures.

6.1 Overall ranking

Table 5. Mean ranks over 15 market–seed evaluations; lower is better. *Wins*: evaluations in which the model has the lowest composite rank — the gradient-trained family takes 10 of 15 composite wins and the econometric family 5. *Fit*: mean wall-clock seconds to train or fit one model; GARCH and GJR-GARCH fit roughly 21,000 $\times$ and 27,000 $\times$ faster than QuantGAN respectively. The shuffled control is scored on every metric but does not compete for rank.

Model	Family	Composite	Fidelity	Temporal	Wins	Fit (s)
GJR-GARCH	econometric	2.524	2.667	2.381	2 / 15	0.018
CNN-WGAN-GP	gradient	2.605	2.429	2.781	6 / 15	180
GARCH	econometric	2.967	3.152	2.781	3 / 15	0.023
QuantGAN	gradient	3.041	3.138	2.943	2 / 15	481
TimeGAN	gradient	3.864	3.614	4.114	2 / 15	81

What it shows. Table ?? gives, for each generator, its mean rank on the two metric families and on their combination, and how often it wins an evaluation outright.

How it is computed. Within each evaluation the five generators are ranked on each of the fourteen ranked metrics of Table ??, rank 1 being closest to the real series. The fidelity rank is the mean over the seven fidelity metrics, the temporal rank the mean over the seven temporal metrics, and the composite rank the mean of the two, so the two families weigh equally however many metrics each contains. The ranks are then averaged over the evaluations.

Why this measure. Ranks rather than raw values, because the metrics live on incommensurable scales. The equal weighting of families is this study’s own design, adopted because an unweighted mean over all metrics is won by a shuffled copy of the data (Section ??).

What this result tells us. GJR-GARCH has the best mean composite rank (2.524); CNN-WGAN-GP trails by 0.081 (2.605). The families split along the metric families: CNN-WGAN-GP leads on fidelity, the marginal distribution, and GJR-GARCH on temporal, the dynamics. Counted per evaluation the picture runs the other way from the mean. Composite wins are CNN-WGAN-GP 6, GARCH 3, GJR-GARCH 2, QuantGAN 2, TimeGAN 2 — the gradient-trained family takes 10 of 15 and the econometric family 5; temporal wins are GJR-GARCH 6, GARCH 3, CNN-WGAN-GP 2, QuantGAN 2, TimeGAN 2; fidelity wins are CNN-WGAN-GP 5, GARCH 3, GJR-GARCH 3, QuantGAN 3, TimeGAN 1. The econometric advantage is therefore in the mean rank and in the dynamics, not in winning most individual evaluations. An unweighted mean over all ranked metrics is kept in the artifacts for comparability and is never used for selection.

6.2 Compute versus performance

Table ??, in Section ??, sets each model’s size and training cost against its composite rank — the one comparison a rank column cannot make. Parameter counts are read from the fitted models and wall-clock is measured around training or fitting. The best-ranked model, GJR-GARCH, has 6 parameters and fits in 0.018 s. The largest, QuantGAN, has 276,481 — a factor of 46,080 — and takes 481 s, a factor of 27,184; it ranks below the smaller model on composite, on fidelity and on temporal.

The ratio is stated and left there. It does not show that parameter count is wasted in general, that the generators would not overtake the baselines at a larger budget, or that they are at their best configuration: no hyperparameter search was run (Section ??). What it shows is that on these data, at this budget, under this evaluation, the additional capacity did not buy additional rank.

6.3 Stylised facts in the generated series

The figures in this section are crops of one figure drawn by the pipeline for BOVESPA, seed 42, set at full page width so their labels can be read. Each shows a single generated series per model: they show what the metrics measure, and the numbers that rank the models are in the tables.

6.3.1 The generated series

Figure 2. One generated series per model beside the real test series, BOVESPA, seed 42: the real series and the three GANs (continued in Figure ??). All rows share one vertical scale, in daily log return. The horizontal axis is a common index of trading days, not common calendar time: real and synthetic series are independent draws, so vertical alignment between rows carries no meaning.



What they show. Figures ?? and ?? plot a synthetic series from each fitted model, the real test series, and the shuffled control.

How they are computed. Each model, fitted on the market’s training split, generates one series with as many observations as the real test split. The control is a random permutation of the real test series. Nothing is smoothed or rescaled.

Why this view. Volatility clustering — large moves arriving in bursts rather than evenly scattered — is the stylised fact visible to the eye (Mandelbrot^[9]; Cont^[7]). The shuffled control is the reference for its absence: it contains exactly the real values, in random order.

What this example tells us. One draw is an illustration, not a statistic. Read with that caveat, the panels show why models can look plausible and still score differently: in this example the TimeGAN path oscillates with an unusually regular pattern, and the QuantGAN path concentrates its largest moves in a single burst. The mode-collapse guard of Section ??, which flags a high lag-1 autocorrelation, is the quantitative counterpart of a visibly regular path.

6.3.2 Marginal distribution: the Q–Q plot

What it shows. Figure ?? compares each model’s distribution of daily returns with the real one, quantile by quantile.

How it is computed. For each probability p from 0.005 to 0.995, the p -quantile of the synthetic series is plotted against the p -quantile of the real series. Identical distributions put every point on the diagonal. A curve steeper than the diagonal in a tail means the model’s tail is wider than the real one; a flatter curve means it is narrower.

Why this measure. The Q–Q plot^[44] compares whole distributions without binning and shows the tails, where risk sits, at the same resolution as the centre. Its ranked counterparts are `quantile_mse` and `wasserstein` (Table ??).

What this example tells us. The shuffled control lies on the diagonal by construction, because it is the real data reordered. That is exactly why a Q–Q plot, like every fidelity metric, cannot on its own tell a good generator from a shuffled deck. The differences are in the tails: here the QuantGAN curve bends far below the diagonal in the lower tail, generating losses several times larger than the real series’ worst days, while the TimeGAN curve flattens and compresses both tails.

6.3.3 Tails: the survival function on log–log axes

What it shows. Figure ?? shows how quickly the probability of a move larger than x decays as x grows.

How it is computed. For each series, absolute returns are sorted and each value x is plotted against the fraction of observations that exceed it. The largest one per cent of order statistics is left off the drawn line, because each is a single-observation estimate of a very small probability; no metric is affected by this.

Why this measure. Tails that decay like a power law are the first stylised fact of returns (Mandelbrot^[9]; Cont^[7]). The log–log survival plot is how that exponent is seen, and the Hill estimator^[38] is how it is measured: `tail_index_diff` ranks models by the gap between the real and synthetic Hill estimates (Section ??).

What this example tells us. A line parallel to the real one has the right tail index; a line that drops away early under-produces extremes; a line that extends to the right produces moves larger than any in the real data. Here the QuantGAN line extends several times beyond the real series’ largest absolute return — the same excess the Q–Q plot shows — while the two GARCH lines stay close to the real one.

6.3.4 Autocorrelation of returns, absolute returns and squared returns

What they show. Figure ?? shows how strongly each series is correlated with its own past, separately for the direction of moves (r_t) and for their size ($|r_t|$ and r_t^2).

How they are computed. At lag k , $\hat{\rho}(k) = \sum_t (x_t - \bar{x})(x_{t+k} - \bar{x}) / \sum_t (x_t - \bar{x})^2$, with x_t equal to r_t , $|r_t|$ or r_t^2 . The ranked metrics `acf_returns_mae`, `acf_absolute_mae` and `acf_squared_mae` are the mean absolute difference between the real and synthetic $\hat{\rho}(k)$ over lags 1 to $\min(50, \lfloor n/4 \rfloor)$, which is 50 at the test-series lengths used here.

Why this measure. Returns themselves are close to uncorrelated, while their absolute and squared values stay positively autocorrelated for many lags. That is volatility clustering (Cont^[7]; Ding, Granger & Engle^[8]), the regularity ARCH and GARCH models were built to describe (Engle^[10]; Bollerslev^[39]). A generator that matches the distribution of returns but not these curves has not learned the dynamics.

What this example tells us. In this test window the real series’ autocorrelation of squared returns stays close to the no-autocorrelation band, so clustering in the real data is weak here, while the QuantGAN and TimeGAN curves sit well above it at short lags: these generators produce more persistent volatility than the data. Section ?? gives the pooled numbers: on `acf_squared_mae` a shuffled copy of the real series beats all 3 GANs.

A discrepancy, reported rather than changed. The MAE values annotated in the figure average lags 1 to 20 and describe themselves as matching the ranked metric. The ranked metric, as the pipeline computes it at these series lengths, averages lags 1 to 50. The ranked values in this report are the pipeline’s. Which window was intended is a question about the metric’s definition, and it has not been changed here.

6.3.5 Rolling volatility

What it shows. Figure ?? shows how the local scale of each series changes over time.

How it is computed. The standard deviation of the preceding 20 daily log returns, recomputed each day, over the first 500 days of each series.

Why this measure. Rolling volatility makes clustering visible as slow swings between calm and turbulent periods — the behaviour conditional-variance models describe (Engle^[10]; Bollerslev^[39]). It is a visual check rather than a ranked metric; its quantitative counterparts are the autocorrelation metrics above and `hurst_diff`.

How to read it. A line that stays flat has no clustering. A line with a single tall spike concentrates its turbulence in one episode. A line whose swings resemble the real one in both height and duration reproduces the clustering the metrics look for.

6.3.6 Moments

What it shows. Figure ?? shows the first four sample moments of each series beside those of the real test series.

How it is computed. The mean, standard deviation, skewness and excess kurtosis of each series' daily log returns.

Why this measure. Location and scale are ranked, through `mean_diff` and `std_diff`. Skewness and kurtosis are shown but never ranked. With tail index α , the moment $\mathbb{E}|r|^k$ exists only for $k < \alpha$ (Hill^[38]); at the tail indices of these markets population kurtosis does not exist, so its sample value grows with the sample instead of converging (Section ??).

What this example tells us. Excess kurtosis differs by an order of magnitude between models — the QuantGAN bar stands far above the real one. That is exactly the kind of difference that looks decisive and is not: with no finite population kurtosis to estimate, a larger sample could move it arbitrarily far.

6.4 The shuffled-control audit

Permutation invariance, counted. The control is the pooled real test series in random order (`numpy.random.default_rng(42).permutation`). It has the identical marginal distribution — same values, same histogram, same moments, same tails — and no temporal structure whatsoever. It is useless for every application synthetic market data is generated for, and it is scored by the full pipeline alongside every generator.

A metric is counted as permutation-invariant in practice when the control's mean value is negligible against the best real generator's on the same metric. On this run that criterion separates cleanly: every metric it selects has a ratio below $3.6\text{e-}05$, and the smallest ratio among the metrics it does not select is 0.36. There is nothing in the gap, so the cut is not a judgement call.

9 of 19 computed metrics are permutation-invariant: `mean_diff`, `std_diff`, `skewness_diff`, `kurtosis_diff`, `wasserstein`, `energy_distance`, `quantile_mse`, `tail_index_diff`, `extreme_events_diff`

Under an unweighted mean over all metrics, the control ranks first — the finding that produced the two-family composite. That is a structural property of any metric defined on the marginal distribution alone, not a quirk of these choices: mean, variance, kurtosis, Wasserstein distance and energy distance all belong to that class. Any evaluation of synthetic financial data built only from distributional metrics cannot distinguish a working generator from a shuffled deck.

Table 6. The shuffled control against the generators on four ordering-sensitive metrics: means over 15 evaluations, lower is better. The control’s value and every generator that beats it are set in bold.

Metric	Control	CNN-WGAN-GP	GARCH	GJR-GARCH	QuantGAN	TimeGAN
ACF(r) MAE	0.0517	0.0511	0.0517	0.0514	0.0548	0.0685
ACF(r^2) MAE	0.0570	0.0590	0.0574	0.0544	0.0806	0.1002
ACF($ r $) MAE	0.0730	0.0610	0.0627	0.0610	0.0981	0.1232
Hurst diff	0.0759	0.0369	0.0397	0.0341	0.0400	0.0881

What it shows. Table ?? compares the control with each generator on the four metrics a shuffled series should fail: the autocorrelation of returns, of absolute returns and of squared returns, and long memory.

How it is computed. Each value is the metric’s mean over all evaluations. The control is scored by exactly the code that scores the generators.

Why this measure. These metrics are ordering-sensitive by construction — they measure volatility clustering (Cont^[7]; Ding, Granger & Engle^[8]) and long memory (Hurst^[23]) — so a series with no dynamics should lose to any generator that has learned some.

What this result tells us. On the autocorrelation of squared returns, the canonical volatility-clustering statistic, the shuffled control beats all 3 GANs, and CNN-WGAN-GP, GARCH, QuantGAN, TimeGAN in all; only GJR-GARCH clears it. On absolute returns the control beats QuantGAN, TimeGAN, and CNN-WGAN-GP, GARCH, GJR-GARCH clear it. On returns it beats GARCH, QuantGAN, TimeGAN, and the models that clear it do so by only 0.0003 to 0.0006. On the Hurst difference it beats TimeGAN alone. A shuffled copy of the real series, with no temporal information at all, outperforms state-of-the-art GAN architectures on the metrics those architectures exist to satisfy.

What this does and does not license. It is tempting to conclude that these ACF metrics are “really” permutation-invariant too, and to move the headline count from 9 to a larger number. We do not, and the reason is the contribution.

These metrics are not permutation-invariant. They are functions of the ordering, and on every row of Table ?? at least one model clears the control — as few as 1 of 5 on ACF(r^2) MAE. A metric that can make that separation is measuring temporal structure. It is doing its job.

What fails is the comparison, not the metric. A metric that separates some models from a shuffled deck and not others is telling us about the models it fails to separate, not about itself. Read the other way round, it is weak evidence about the metric.

Therefore: permutation-invariance in practice is a property of the comparison set, not of a metric in isolation. The same metric is discriminating against one model and uninformative against another, in the same run, on the same data. A count of “how many metrics are permutation-invariant” is only meaningful once the set of models being compared is fixed — and reporting one without the other is how a suite comes to look more discriminating than it is.

We keep the headline at 9 of 19: the metrics that are invariant by construction. The ACF results are reported as what they are: 4 of the 5 generators lose to the control on at least one ordering-sensitive metric, on the metrics most often used to claim success — and FIDELITY_COLS and TEMPORAL_COLS are unchanged.

The practical recommendation follows: a shuffled control belongs in every evaluation table for synthetic

time series, permanently, in the role a positive control plays in a biology experiment. It costs one permutation and it is the only line in the table that cannot be gamed by learning the marginal.

6.5 Walk-forward discriminability

Table 7. Discriminative AUC over every walk-forward fold of every evaluation. $|AUC - 0.5|$ is the ranked quantity; z is the distance of the mean AUC from the empirical null of 0.506 ± 0.084 , in null standard deviations.

Model	AUC (mean $\pm$ sd)	$ AUC - 0.5 $	z vs null	Folds
CNN-WGAN-GP	0.637 ± 0.136	0.137	1.56	75
GARCH	0.683 ± 0.167	0.183	2.10	75
GJR-GARCH	0.655 ± 0.164	0.155	1.78	75
QuantGAN	0.741 ± 0.175	0.241	2.80	75
TimeGAN	0.814 ± 0.144	0.314	3.66	75

Table 8. Mean discriminative AUC by walk-forward fold, over all models and markets. The training window grows with the fold.

Fold	Training length	Mean AUC	sd
0	810–837	0.811	0.154
1	1,620–1,669	0.692	0.135
2	2,430–2,501	0.747	0.147
3	3,240–3,333	0.616	0.177
4	4,050–4,165	0.664	0.166

What they show. Tables ?? and ?? show whether a classifier can tell each model’s output from real data out of sample, and how that changes as the training window grows.

How they are computed. In each of the 5 walk-forward folds (Section ??) a freshly trained model generates a series the length of the fold’s test block. A logistic regression then tries to separate real from synthetic 20-day windows described by their mean, standard deviation, mean absolute return and mean squared return, scored by five-fold cross-validation without shuffling. An AUC of 0.5 means the two are indistinguishable.

Why this measure. A classifier-based discriminative score is the indistinguishability test of TimeGAN^[4], adopted by CTBench^[16]. Walk-forward evaluation keeps test data strictly after training data, as time-series evaluation requires (Tashman^[13]; Bergmeir & Benítez^[14]). The null is not exactly 0.5: over 15 real-versus-real half-splits it was 0.506 ± 0.084 , a prior measurement, so each AUC is read as a distance from that null.

What this result tells us. 3 of 5 models sit more than 1.96 null standard deviations from it (TimeGAN, QuantGAN, GARCH). The rest (CNN-WGAN-GP, GJR-GARCH) lie within that distance on the fold-averaged mean, so on this metric alone they cannot be distinguished from a perfect generator at the 5% level; 5 of 5 mean AUCs lie above the null mean. AUC falls as the training window grows, from 0.811 at fold 0 (training on 810–837 observations) to 0.664 at fold 4 (4,050–4,165), but the decline is not monotonic: fold(s) 2, 4 rise above the fold before. Fold index, training length and the calendar period a fold covers all advance together, so this design cannot separate “more training data helps” from “the later period is easier to imitate”.

6.6 Tail index

Table 9. `tail_index_diff`, the absolute difference between the real and synthetic Hill tail-index estimates, over all evaluations.

Model	Mean	sd	Observed range
CNN-WGAN-GP	0.943	0.831	0.208–2.871
GARCH	1.195	1.062	0.086–3.785
GJR-GARCH	1.316	0.856	0.019–2.385
QuantGAN	1.079	0.892	0.146–3.211
TimeGAN	2.682	3.330	0.024–9.825

What it shows. Table ?? shows how far each model’s tail heaviness is from the real series’, and how much that varies across evaluations.

How it is computed. The Hill estimator^[38] takes the absolute returns above their 95th percentile u , $X_1, \dots, X_k$, and estimates the tail index as $\hat{\alpha} = \left[\frac{1}{k} \sum_i \log(X_i/u) \right]^{-1}$; an estimate above 20 means the series has effectively no tail and counts as a failure. The metric is $|\hat{\alpha} - \tilde{\alpha}|$.

What this result tells us. TimeGAN sits at 2.68 against 0.94–1.32 for the other four — a real and large gap. But the dispersion swamps the comparison between the rest: the spread between model means is 1.74, and the largest standard deviation within one model is 3.33, which exceeds it. Market-to-market and run-to-run variation on this metric is larger than most differences between models, so a single-run winner on `tail_index_diff` is not a finding, and none is reported.

6.7 Mode collapse

The pipeline’s post-generation guard checks every generated series and prints a [WARNING] when the lag-1 autocorrelation exceeds |0.1| — a return series should have almost none, and a high value means a smooth path rather than returns.

In this run the guard fired **18 times**, across 2 model(s) — TimeGAN (16), QuantGAN (2). Mean $|\text{ACF}(1)| = 0.533$, over the range -0.622 to 0.942 ; generated sd 0.0002–0.0440. Read from `run_log_20260912_131816.log`.

The ranking agrees. TimeGAN is the worst of the five on **all 7 of 7 temporal metrics**, on 5 of 7 fidelity metrics, and on 14 of the 19 computed metrics overall. It is *not* worst on `std_diff` (QuantGAN), `skewness_diff` (QuantGAN), `quantile_mse` (QuantGAN), `arch_pvalue_diff` (GARCH), `arch_stat_diff` (QuantGAN) —

the model in brackets is worse there — so “worst on every metric” would be an overstatement, and the scope is stated instead.

An earlier explanation is withdrawn. A previous draft attributed this to tanh saturation in the Recovery network. This run’s own [DIAG] latent instrumentation does not support that: across 90 latent readings saturation never exceeds 3.1% against the 50% threshold the diagnostic exists to detect, and across 90 reconstruction readings output sd never differs from target sd by more than 0.0014, so the autoencoder is functioning. The collapse is real and the mechanism is open.

6.8 GARCH fits at the integrated boundary

What it shows. How often a GARCH or GJR-GARCH fit converged to a persistence of 1 — the point at which the conditional variance never reverts to a long-run level — and what the pipeline did about it.

How it is computed. Persistence is $\alpha + \beta$ for GARCH(1,1)^[39] and $\alpha + \beta + \gamma/2$ for GJR-GARCH(1,1)^[40], arch’s own definition. Every econometric fit — the main fit and one fit per walk-forward fold, for both models — records its unconstrained persistence. A fit within 10^{-6} of 1 is refitted under the bound persistence $\leq 1 - 10^{-4}$, and its output is generated from the constrained parameters.

Boundary fits, counted. Every econometric fit records its unconstrained persistence as a result: the main fit on each training split and one per walk-forward fold, for both GARCH variants. Across 180 fits (60 market–window–model combinations $\times$ 3 seeds; a fit does not depend on the seed), **12 reached the integrated boundary** — persistence within 10^{-6} of 1 — in 4 combinations, on NIFTY50, SHANGHAI. The constraint was applied to exactly those fits and to no other.

Market	Window	Model	Unconstrained	Constrained	Seeds
NIFTY50	fold 0	GARCH	1.0000000000	0.9999000000	3
SHANGHAI	main	GARCH	1.0000000000	0.9999000000	3
SHANGHAI	fold 3	GARCH	1.0000000000	0.9999000000	3
SHANGHAI	fold 3	GJR-GARCH	1.0000000000	0.9999000000	3

Boundary fits sit within 3.5×10^{-13} of 1; the largest persistence among the other 168 fits is 0.99987666 ($1 - p = 1.23 \times 10^{-4}$). Constrained persistence is 0.9999000000–0.9999000000, at a log-likelihood change of -0.019743 to -0.000581 — the constrained fit is statistically indistinguishable from the unconstrained one, which is what makes $\delta = 10^{-4}$ a calibrated choice rather than an arbitrary one.

The hits are seed-invariant. Each combination above appears in all 3 of 3 seeds, and the log-likelihood cost of the refit is identical across seeds (largest spread 0.000000). A fit by maximum likelihood on fixed data cannot depend on the seed, so this is what should happen — and it establishes the hits as a property of the data window rather than of estimation noise, which is what licenses reading them as a statement about the data rather than about the estimator.

Two distinct causes. 3 of the affected combinations sit on long windows (3,240–3,888 observations, against a median fitted window of 2,870): SHANGHAI main GARCH; SHANGHAI fold 3 GARCH; SHANGHAI fold 3 GJR-GARCH. These are the structural-break mechanism — breaks in the unconditional variance absorbed as persistence (Lamoureux & Lastrapes 1990^[42]; Mikosch & Stărică 2004^[43]). SHANGHAI over 2006–2026 spans the 2007 bubble and crash, 2015 and COVID: three volatility regimes inside one training window. 1 sits on a short window (829 observations, the shortest walk-forward fold): NIFTY50 fold 0 GARCH. That is a small-sample effect — at that length the data does not separate slow mean reversion in variance from its absence — and is reported separately: only the first group is evidence about market structure. At $\alpha + \beta = 1$ the conditional variance is a martingale with no unconditional variance to revert to, so simulated paths random-walk without an anchor.

The two-stage fit, and why it is done at fit time. **Two-stage fit.** Stage 1 fits by maximum likelihood without further constraint and always records persistence ($\alpha + \beta$, plus $\gamma/2$ for GJR-GARCH — arch’s own definition). Stage 2 runs only when stage 1 lands within 10^{-6} of 1: it refits under persistence $\leq 1 - \delta$, $\delta = 10^{-4}$, and output is generated from the constrained parameters; both parameter sets are stored with the fit. The refit tightens the right-hand side of arch’s own stationarity constraint and re-optimises arch’s own likelihood, so nothing is reimplemented, and on stationary windows it reproduces the unconstrained fit. δ sits in an observed gap: across 110 refits, 6 were within 1.7×10^{-13} of 1 and every other fit at least 1.23×10^{-4} below it, so the bound binds only on fits that were already at the boundary.

Why at fit time. Whether a boundary fit explodes is arbitrary. In the preceding run, GARCH exploded on SHANGHAI fold 3 for all three seeds while GJR-GARCH, on the same window, did not; local refits reversed it, 200 of 200 simulated paths against 0 of 200. Rejecting explosive draws after generation would keep whichever draws happened to survive; constraining the fit removes the knife-edge. A post-generation guard still raises if output sd leaves 1/10–10× the training sd. It is not guaranteed silent: at persistence 0.9999 one simulated path in 200 per boundary window exceeded 10× in local refits (max 20.8×), though the pipeline’s seeds stayed within 1.7×. The refit counts and the knife-edge are prior measurements recorded in the repository’s standing constraints, not artifacts of this run.

What it prevented. In the run that preceded this procedure, the unconstrained fit on the same SHANGHAI fold 3 window generated walk-forward Wasserstein of 74.795, 7.814 and 43.006 across the three seeds — against a run-wide median of 0.0025 — with discriminative AUC of exactly 1.000 in all three (a prior measurement; those artifacts are superseded). In this run the largest walk-forward Wasserstein over all 150 econometric folds is 0.01427 against a median of 0.00227, and 0 of them reach AUC 1.000.

6.9 Downstream utility

What it shows. Table ?? asks whether a risk model built on synthetic data works on real data — the use synthetic market data is wanted for.

Table 10. Train on synthetic, test on real: a GARCH(1,1) fitted to each model’s synthetic series forecasts the variance of real returns. Medians and means over 3 pooled backtests; lower QLIKE is better. *Fail*: backtests with a degenerate, positive QLIKE.

Model	QLIKE med.	QLIKE mean	Cov. err.	Kupiec p	Viol.	n	Fail
CNN-WGAN-GP	-8.134	-8.1	0.0020	0.644	129	2,479	—
GJR-GARCH	-8.130	-8.1	0.0068	0.110	136	2,479	—
QuantGAN	-8.124	-8.0	0.0008	0.857	122	2,479	—
GARCH	-8.106	-8.1	0.0113	0.012	138	2,479	—
TimeGAN	-8.102	-8.0	0.0012	0.779	127	2,479	—
CONTROL: shuffled real	-8.031	-8.0	0.0060	0.160	109	2,479	—

How it is computed. For each seed, the real test series of every market are concatenated, and so are each model’s per-market synthetic series, in the same order. A Gaussian GARCH(1,1) is fitted to the pooled synthetic series and its parameters are run over the pooled real returns to give one-step variance forecasts σ_t^2 and a 5% value-at-risk. QLIKE, $\frac{1}{n} \sum_t (\log \sigma_t^2 + r_t^2 / \sigma_t^2)$, scores the variance forecasts; the Kupiec test checks the rate of VaR violations and the Christoffersen test their independence.

Why this measure. QLIKE is robust to the noise in squared returns as a proxy for variance (Patton^[37]); the Kupiec^[34] and Christoffersen^[35] tests are the standard backtests of a value-at-risk model.

What this result tells us. CNN-WGAN-GP has the lowest median QLIKE (−8.134); backtest by backtest, the lowest QLIKE goes to CNN-WGAN-GP 2, GARCH 1. **Median and mean are both shown.** No model-seed cell returned a positive QLIKE in this run; the largest gap between a model’s mean and median QLIKE is 0.125 (TimeGAN).

On n . Each backtest runs once per seed over the pooled test set: the real test series of all 5 markets (BOVESPA, FTSE, MOEX, NIFTY50, SHANGHAI) concatenated in market order, with each model’s per-market synthetic draws concatenated in the same order. n is 2,479 per backtest, over 3 seeds, so each row above summarises 3 pooled backtests. That n meets the $n \gtrsim 600$ at which QLIKE ranks stably, and is close to the $n \approx 2,480$ at which Kupiec power was measured at $\approx 99\%$. Concatenating 5 markets means one GARCH filter crosses 4 market boundaries within 2,479 pooled observations. The shuffled control’s QLIKE beats a genuine generator’s in 2 of 15 control-versus-generator comparisons (13%).

ARCH-LM: a regime failure. An 80-replication Monte Carlo (GARCH(1,1), $n = 1,200$, Gaussian innovations) found `arch_pvalue_diff` identifies the better-fitting model 99% of the time and `arch_stat_diff` 76%, the latter with null sd ≈ 46.4 — two draws from the same process gave LM 56.8 and 124.8.

On real BRICS data the pvalue variant saturates completely: real and shuffled series both underflow to $p = 0.0$, so $|\Delta p| = 0.000000$ and the metric rates the adversarial control as a perfect match. The stat variant detects the control but is too noisy to rank on.

Neither variant is reliable across both regimes, so both are descriptive-only. ACF-MAE is the primary volatility-clustering ranking signal: continuous, no saturation regime, 88% accuracy in the same Monte Carlo. This is an empirical result about a standard test, not an implementation note.

6.10 Metric circularity

Stated before a referee states it. `resid_kurtosis_diff` fits a GARCH(1,1) to each series, extracts $\varepsilon_t = r_t / \sigma_t$, and compares the excess kurtosis of those residuals. Two of the five generators *are* GARCH models, so they are being scored, in part, by their own model class.

This is not fatal, and the reason is what the metric asks. It does not ask “does a GARCH fit this series” — that would be circular outright. It asks whether the tails remain heavy *after* GARCH has removed what it can explain, which is Cont’s stylized fact 7^[7] and a property a GARCH-*t* generator can fail. The measured values bear this out: the econometric models score GARCH 3.12, GJR-GARCH 3.04, against 2.64–4.02 for the GANs and 0.94 for the shuffled control — neither scores better than the shuffled control on it.

It remains an advantage of degree, and it is one reason `composite_rank` is reported alongside its two components rather than alone — `resid_kurtosis_diff` is one of seven temporal metrics, and the temporal result does not rest on it. Gaussian QMLE is used for the fit rather than Student-*t* precisely to avoid the stronger circularity: a *t* specification absorbs the excess kurtosis by construction and makes the test uninformative for every model.

6.11 What the budget choice cost

Budget parity fixes which models are compared fairly. It does not fix *at what budget*, and that choice moves the answer.

At a smoke-test budget of 1,000 generator updates, QuantGAN won all five repeat runs on `composite_rank` (1.429–1.500), with CNN-WGAN-GP at 1.786–1.893 — a prior measurement recorded in the repository’s standing constraints, not reproducible from this run’s artifacts.

At 9,000 generator updates, ordering the 3 gradient-trained models by `composite_rank` within this five-model ranking gives CNN-WGAN-GP 10 / 15, QuantGAN 3 / 15, TimeGAN 2 / 15. CNN-WGAN-GP leads.

The winner changed. Ranking at a smoke-test budget did not predict ranking at full budget. This is the budget-parity lesson one level up: even with parity correctly enforced, the budget you evaluate at determines the result.

Most papers in this literature report a single budget without justifying the choice. We report ours — and state plainly that 9,000 updates is itself a choice whose sensitivity we have only partially characterised. Two points on a budget curve is not a budget curve. A benchmark that reports one point is reporting a result conditional on an unstated hyperparameter.

6.12 Run-to-run nondeterminism

At fixed seed and identical code, five repeated runs show sharply different stability by architecture. These figures are a prior measurement on the earlier three-GAN configuration, recorded in the repository’s standing constraints; they were not re-measured in this run. TimeGAN and the shuffled control are bit-identical across all five — `sd` = 0 on every metric, every fold. CNN-WGAN-GP drifts slightly: Wasserstein CV 1.6%, walk-forward AUC `sd` 0.009. QuantGAN drifts substantially: Wasserstein CV 37%, `tail_index_diff` CV 79%, raw AUC 0.551–0.734.

This is not a seeding bug. Loss traces agree to four significant figures at epoch 1 and separate by epoch 5, consistent with QuantGAN being the only model combining WGAN-GP’s double-backward gradient penalty with dilated convolutions.

`torch.use_deterministic_algorithms(True)` and `torch.backends.cudnn.deterministic` remain deliberately unset. The consequence is now measured rather than assumed, and that is the justification: `composite_rank` is stable across runs, but `tail_index_diff`, `hurst_diff` and `mean_diff` change their winning model between identical repeat runs. That is why no single-run winner on those metrics appears anywhere in this report.

7 Discussion

7.1 A small model ranks first, and why that is the interesting part

The result is not that deep generative models fail. CNN-WGAN-GP leads the field on `fidelity_rank` and the gradient family takes 9 of 15 fidelity wins; these models reproduce the marginal distribution of BRICS returns well, including tails that defeat a Gaussian by orders of magnitude.

The result is that reproducing the marginal is not the hard part, and it is not the part that transfers. On `temporal_rank` — volatility clustering, long memory, conditional heavy tails, indistinguishability — the econometric family takes 9 of 15, and GJR-GARCH finishes first overall on the equal-weighted composite. The per-cell composite count goes the other way (gradient 10 of 15), and CNN-WGAN-GP trails by only 0.081: the econometric edge is in the mean rank and in the dynamics, not a sweep.

Two readings are available and the data does not settle between them. Either the GAN architectures have not been given enough budget, capacity or tuning to reach their potential here — no hyperparameter search was run, and the budget-sensitivity result shows the budget matters — or conditional heteroskedasticity with a leverage term is simply a very good model of daily equity returns, and a general-purpose sequence generator learning it from scratch on $\approx 4,000$ observations per market is at a structural disadvantage. Both are consistent with what we measured. What is not consistent with what we measured is a benchmark that omits the parametric baseline and reports the best GAN as the state of the art.

This is why the econometric arm was added. A benchmark of GANs against GANs answers “which GAN”, and produces a winner regardless of whether any entrant is useful.

7.2 The audit as a methodological contribution

The shuffled control is the cheapest useful thing in this pipeline: one permutation of the test set, scored by the same code as everything else.

It establishes two things. First, 9 of 19 computed metrics score it perfectly by construction, so any composite that mixes families without balancing them is won by a sequence with no temporal content. The fix — taxonomise metrics by invariance, weight families rather than metrics, exclude the control from rank competition — needs no additional computation.

Second, and less comfortably: on the ordering-sensitive metrics the control still outperforms genuine GAN generators. That is not a metric failure. It is a measurement of the generators, made possible only because a control was present to make it against.

The general form of the caveat is the part we would ask others to adopt. **Whether a metric discriminates is a joint property of the metric and the model set.** The same statistic separates GARCH from a shuffled deck and fails to separate three GANs from it, in one run, on one dataset. Reporting a suite’s discriminating power without naming the models it was measured against overstates it.

7.3 Research directions

Four questions follow directly from the results above. Each is stated with the measurement it would take to answer it.

A metric suite validated against adversarial controls by construction. The shuffled control audits one failure: blindness to ordering. It was added after the fact, and it found that 9 of 19 metrics fail it. The direction is to reverse that order and admit a metric to the ranked suite only after it has been shown to separate the real series from a fixed battery of controls, each of which destroys exactly one property: a random permutation (destroys all dependence), a stationary block bootstrap (keeps short-range dependence, destroys long memory), phase-randomised surrogates that keep the linear autocorrelation and destroy nonlinear dependence such as volatility clustering (Theiler et al.^[45]), and a GARCH-filtered residual shuffle (keeps conditional variance, destroys conditional tails). For each metric and control the quantity to report is the separation at the pooled test length, $n \approx 2,480$, against its real-versus-real null. The output would be a table stating, for each metric, which properties it can detect — the permutation-invariance count generalised to every property the fidelity and temporal families claim to measure.

Budget dependence of the ranking. Among the gradient-trained models the ordering at a smoke-test budget of 1,000 generator updates was reversed at 9,000 (Section ??). Two budgets cannot say whether the ranking converges, keeps changing, or reverses again. The experiment is a budget sweep at fixed seeds — for example 1,000, 2,000, 4,000, 9,000 and 18,000 generator updates — reporting each model’s composite rank as a function of budget together with its across-seed spread. The question it answers is whether a single-budget benchmark in this literature is reporting a ranking or a point on a curve. The run-to-run nondeterminism of QuantGAN (Section ??) sets how many seeds each point needs.

Conditional generation under regimes. Every model here is unconditional: it generates returns without being told whether the market is calm or in crisis, and the crisis episodes in twenty years are few. A regime-conditional generator would take a label — for instance the tercile of trailing 20-day realised volatility — and would be scored within each regime, with the same metrics computed separately on the calm and turbulent days of the real test series. The GARCH baselines have a natural conditional counterpart: filter the variance up to the conditioning date, then simulate forward. The test is whether a network conditioned on regime closes the temporal-rank gap to the econometric models, which carry the conditioning implicitly in σ_t^2 . The MOEX suspension of 2022 is a candidate held-out regime.

Whether the six-parameter win holds across asset classes. GJR-GARCH’s leverage term encodes the gain–loss asymmetry of equity indices^[40]. The same pipeline run unchanged on asset classes where that structure is weaker or different — major currency pairs, where the leverage effect is small; commodity futures, with jumps and seasonality; and government bond yields, with regime-dependent volatility — would show whether the econometric result is a property of the model class or of equity returns. The pipeline needs no change beyond the data files; what it would measure is the composite rank and the temporal rank of each family, asset class by asset class.

8 Limitations

No hyperparameter search. All three GAN architectures use reference-implementation defaults from their original papers. No search over learning rate, hidden dimension or noise dimension has been run (Bergstra & Bengio 2012^[12] would be the protocol). The econometric baselines have no comparable free parameters, so this asymmetry favours the baselines and is stated as such. It is the largest single caveat on the headline result.

One budget, partially characterised. See Section ??: the winner among the GANs changes between a smoke-test budget and 9,000 updates. Two points do not characterise a curve.

No diffusion baseline. Takahashi & Mizuno (2025)^[29] report diffusion-based generators outperforming GANs on several stylized-fact metrics. This benchmark does not include one. Reviewers will ask; the answer is not yet.

Fold confound unseparated. Fold index, training length and historical period advance together (Table ??).

Downstream QLIKE outliers. No pooled backtest returned a degenerate QLIKE in this run.

Crisis regimes remain few. Twenty years resolves the estimator problem — Hill sd falls from ≈ 0.52 at $n = 995$ to ≈ 0.035 at $n \approx 4,960$ — but not the sparsity of independent crisis episodes. The extreme days in this dataset cluster into a handful of regimes: 2008, 2020, and the 2022 MOEX shock. A benchmark validated against a few crisis episodes is validated against a few crisis episodes, however many market-days each contributes.

Determinism unenforced. Deliberate, and measured (Section ??); the consequence is that no single-run winner is reported on the three metrics whose winner changes between identical runs.

Sampling cost is not recorded. The run measures training and fitting time but not the time a fitted model takes to generate a series, so the cost comparison in Table ?? covers training only.

Figures come from one evaluation. Every figure shows BOVESPA, seed 42. The tables pool all 15 evaluations; the figures do not, and their visual impressions are illustrations rather than findings.

Figure text is fixed at render time. The figures are raster images drawn by the pipeline at a fixed size. They are set at full text width here, which makes the single-panel figures legible, but the series-row figures carry seven rows each and their axis labels remain small. Re-drawing them at print size in the pipeline would fix this; the report cannot.

The ACF annotation window differs from the ranked metric. The mean absolute errors annotated in Figure ?? average lags 1–20; the ranked metrics average lags 1–50 (Section ??). Which was intended is a question about the metric definition and has not been changed.

Some evidence is prior measurement. The empirical AUC null, the repeated-run nondeterminism figures, the smoke-budget ranking and the Monte Carlo accuracy of the ARCH-LM variants were measured before this run and are recorded in the repository’s documentation, not in this run’s artifacts. They are marked as prior where they are used.

The guard log is not tracked. The mode-collapse count is read from the run log in the reports directory. The run’s data artifacts are version-controlled; that log is not written alongside them.

9 Next steps

In order of the value of the information each would add:

1. **Budget curve.** Evaluate every gradient-trained model at several budgets and report rank against budget (Section ??). The observed reversal makes this the most informative next run, ahead of adding models.
2. **Hyperparameter search.** Random search^[12] of 20–30 candidates per architecture, scored on composite rank, with the best candidate retrained at the full budget. This tests directly whether the generators’ temporal deficit is architectural or a matter of configuration.
3. **Fixed-length rolling folds.** Walk-forward folds of equal training length on a rolling window, to separate training length from historical period (Section ??).
4. **Adversarial-control battery.** Add the block-bootstrap, phase-randomised and residual-shuffle controls beside the permutation control and report each metric’s separation from each.
5. **Record sampling cost and resolve the ACF window.** Time generation alongside training; decide whether the ACF metrics average lags 1–20 or 1–50, and align figure and metric.
6. **A diffusion baseline**^[29], scored by the same pipeline, since diffusion generators are reported to outperform GANs on several stylised-fact metrics and this benchmark has none.

References

- [1] Goodfellow, I. et al. (2014). Generative adversarial nets. *NeurIPS* 27, 2672–2680.
- [2] Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein generative adversarial networks. *ICML*, PMLR 70, 214–223.
- [3] Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. (2017). Improved training of Wasserstein GANs. *NeurIPS* 30.
- [4] Yoon, J., Jarrett, D., & van der Schaar, M. (2019). Time-series generative adversarial networks. *NeurIPS* 32, 5508–5518.
- [5] Wiese, M., Knobloch, R., Korn, R., & Kretschmer, P. (2020). Quant GANs: deep generation of financial time series. *Quantitative Finance* 20(9), 1419–1440.
- [6] Adams, Z., Füß, R., & Glück, T. (2019). Are correlations constant? *Financial Management* 48(3), 793–826.
- [7] Cont, R. (2001). Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance* 1(2), 223–236.
- [8] Ding, Z., Granger, C. W. J., & Engle, R. F. (1993). A long memory property of stock market returns and a new model. *Journal of Empirical Finance* 1(1), 83–106.
- [9] Mandelbrot, B. (1963). The variation of certain speculative prices. *The Journal of Business* 36(4), 394–419.
- [10] Engle, R. F. (1982). Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica* 50(4), 987–1007.
- [11] Székely, G. J., & Rizzo, M. L. (2004). Testing for equal distributions in high dimension. *InterStat* 5(16.10).
- [12] Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *JMLR* 13, 281–305.
- [13] Tashman, L. J. (2000). Out-of-sample tests of forecasting accuracy. *International Journal of Forecasting* 16(4), 437–450.
- [14] Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences* 191, 192–213.
- [15] Ang, Y., Bansal, P., Gupta, G., & Su, L. (2023). TSGBench: time series generation benchmark. *PVLDB* 17(3), 305–318.
- [16] Liao, Q. et al. (2025). CTBench: a comprehensive benchmark for conditional time series generation. *NeurIPS* 38, Datasets and Benchmarks Track.
- [17] Gatheral, J., Jaisson, T., & Rosenbaum, M. (2018). Volatility is rough. *Quantitative Finance* 18(6), 933–949.
- [18] Cont, R., & Das, S. (2024). Rough volatility: fact or artefact? *Sankhya B* 86(1), 191–223.
- [19] Cho, K. et al. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. *EMNLP*, 1724–1734.
- [20] Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv:1803.01271*.
- [21] Radford, A., Metz, L., & Chintala, S. (2015). Unsupervised representation learning with deep convolutional GANs. *arXiv:1511.06434*.
- [22] Lobato, I. N., & Savin, N. E. (1998). Real and spurious long-memory properties of stock-market data. *Journal of Business & Economic Statistics* 16(3), 261–268.
- [23] Hurst, H. E. (1951). Long-term storage capacity of reservoirs. *Transactions of the ASCE* 116, 770–799.
- [24] Mandelbrot, B., & Wallis, J. R. (1969). Robustness of the rescaled range R/S. *Water Resources Research* 5(5), 967–988.
- [25] Ma, J. et al. (2024). GenTS: generative time series via large language models. *arXiv preprint*.
- [26] Akiba, T. et al. (2019). Optuna: a next-generation hyperparameter optimization framework. *KDD 2019*,

2623–2631.

- [27] Meldrum, A. et al. (2025). Deep generative models for financial time series: a comparative study. *arXiv:2510.26076*.
- [28] SFAG Team. (2026). Synthetic financial asset generator. *arXiv:2601.12990*.
- [29] Takahashi, S., & Mizuno, T. (2025). Diffusion-based generation of financial time series. *Quantitative Finance* 25(10).
- [30] Gruver, N., Finzi, M., Qiu, S., & Wilson, A. G. (2023). Large language models are zero-shot time series forecasters. *NeurIPS* 36.
- [31] Ansari, A. F. et al. (2024). Chronos: learning the language of time series. *arXiv:2403.07815*.
- [32] Hamdouche, M. et al. (2025). Forging time series: a comprehensive study of GAN-based synthetic financial data. *arXiv:2505.17103*.
- [33] LeBaron, B. (2001). Stochastic volatility as a simple generator of apparent financial power laws and long memory. *Quantitative Finance* 1(6), 621–631.
- [34] Kupiec, P. H. (1995). Techniques for verifying the accuracy of risk measurement models. *Journal of Derivatives* 3(2), 73–84.
- [35] Christoffersen, P. F. (1998). Evaluating interval forecasts. *International Economic Review* 39(4), 841–862.
- [36] Bollerslev, T. (1987). A conditionally heteroskedastic time series model for speculative prices and rates of return. *Review of Economics and Statistics* 69(3), 542–547.
- [37] Patton, A. J. (2011). Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics* 160(1), 246–256.
- [38] Hill, B. M. (1975). A simple general approach to inference about the tail of a distribution. *The Annals of Statistics* 3(5), 1163–1174.
- [39] Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics* 31(3), 307–327.
- [40] Glosten, L. R., Jagannathan, R., & Runkle, D. E. (1993). On the relation between the expected value and the volatility of the nominal excess return on stocks. *The Journal of Finance* 48(5), 1779–1801.
- [41] Vuletić, M., Prenzel, F., & Cucuringu, M. (2024). Fin-GAN: forecasting and classifying financial time series via generative adversarial networks. *Quantitative Finance* 24(2), 175–199. Related work: a conditional GAN for return forecasting and classification with an economics-driven loss — a different model from the unconditional CNN-WGAN-GP generator benchmarked here, despite the similar name.
- [42] Lamoureux, C. G., & Lastrapes, W. D. (1990). Persistence in variance, structural change, and the GARCH model. *Journal of Business & Economic Statistics* 8(2), 225–234.
- [43] Mikosch, T., & Stărică, C. (2004). Nonstationarities in financial time series, the long-range dependence, and the IGARCH effects. *Review of Economics and Statistics* 86(1), 378–390.
- [44] Wilk, M. B., & Gnanadesikan, R. (1968). Probability plotting methods for the analysis of data. *Biometrika* 55(1), 1–17.
- [45] Theiler, J., Eubank, S., Longtin, A., Galdrikian, B., & Farmer, J. D. (1992). Testing for nonlinearity in time series: the method of surrogate data. *Physica D* 58, 77–94.

Glossary

Stylised facts

Statistical regularities shared by returns across markets and periods: heavy tails, near-zero autocorrelation of returns, volatility clustering, long memory in volatility, gain–loss asymmetry and others (Cont^[7]). Appendix ?? maps each to the metric that tests it.

Tail index

The exponent α in $P(|r| > x) \propto x^{-\alpha}$ for large x . Smaller α means heavier tails. The moment $\mathbb{E}|r|^k$ is finite only for $k < \alpha$, so with α between 2 and 4 the variance exists but the kurtosis does not.

Hill estimator

The standard estimator of the tail index^[38]: with u a high threshold and $X_1, \dots, X_k$ the observations above it, $\hat{\alpha} = \left[\frac{1}{k} \sum_i \log(X_i/u) \right]^{-1}$. Here the threshold is the 95th percentile of $|r_t|$.

Integrated GARCH

A GARCH model whose persistence $\alpha + \beta$ (plus $\gamma/2$ for GJR) equals 1. Shocks to the conditional variance then never decay and there is no long-run variance to revert to. Fits reach this boundary when the sample contains structural breaks in volatility^{[42][43]}.

QLIKE

A loss function for variance forecasts, $\frac{1}{n} \sum_t (\log \sigma_t^2 + r_t^2 / \sigma_t^2)$. It ranks forecasts consistently even though the squared return is only a noisy proxy for the true variance (Patton^[37]). Lower is better; it can be negative.

Kupiec and Christoffersen tests

Backtests of a value-at-risk model. The Kupiec test^[34] checks whether violations — days the loss exceeds the VaR — occur at the nominal rate; the Christoffersen test^[35] checks whether they are independent rather than clustered. Both are likelihood-ratio tests; a small p -value rejects the model.

Walk-forward validation

Evaluation of a time-series model on test blocks that always follow its training data in time, retraining as the training window moves forward^[13]. It avoids the leakage random cross-validation causes when observations are serially dependent^[14].

Discriminative AUC

The area under the ROC curve of a classifier trained to tell real from synthetic windows^[4]. 0.5 means the classifier cannot separate them; 1.0 means it separates them perfectly. The ranked quantity is the distance from 0.5.

Permutation invariance

A metric is permutation-invariant if its value does not change when the observations are put in a different order. Every metric defined on the marginal distribution alone is permutation-invariant, so it scores a shuffled copy of the real data as a perfect match.

Mode collapse

A failure of GAN training in which the generator produces a narrow range of outputs — in this setting, a near-constant or smooth path rather than a return series — because that output fools the discriminator well enough.

Gradient penalty

The term $\lambda \mathbb{E}[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2]$ added to the critic loss of a Wasserstein GAN, evaluated at random interpolations between real and generated samples. It keeps the critic approximately 1-Lipschitz, as the Wasserstein objective requires (Gulrajani et al.^[3]). Here $\lambda = 10$.

n_{critic}

The number of critic updates per generator update in WGAN-GP training. The critic must approximate the Wasserstein distance before its gradient is useful to the generator; $n_{\text{critic}} = 5$ is the standard setting^[3] and is used here.

A Record of significant fixes

Table ?? records the defects found in the pipeline that changed, or would have changed, reported results, and what was done about each. *Evidence* names where the record of the defect lives: *computed* marks a quantity this report reads from the run’s artifacts; *prior* marks a measurement taken before this run and recorded in the repository’s documentation or code, not in this run’s artifacts.

Table 11. Significant fixes. Every change to data handling, training or evaluation below was made before the run reported here; the report’s numbers are from the corrected pipeline.

Defect	What was wrong, and its effect	Resolution	Evidence
TimeGAN batch loop, and the 492× budget gap	TimeGAN’s training loop drew one batch per epoch instead of iterating over all of them: with 3,835 windows and batch size 128, one gradient step per epoch where 29 were due. Separately, a smoke-test override of epochs reached two models and not the third, so the models received generator-update budgets differing by a factor of 492. The under-trained model read as an architectural failure until the step counts were printed.	Inner batch loop added. Budgets specified in gradient steps, declared in one configuration cell, with generator-update parity asserted at run time (Section ??).	Prior: knowledge base ch. 10; notebook cell 20 comment; step-count printout retained.
Min-max normalisation	With tail indices of 2.5–3.2 one extreme day sets the scale: min-maxing MOEX to $[-1, 1]$ leaves 98% of the data in 14.3% of the range, median +0.233, so the generator must learn an offset before any dynamics.	z-score followed by $\tanh(z/3)$, inverted by $3 \operatorname{arctanh}(y)$. Changing only this moved QuantGAN’s output sd from 3.7× to 1.2× real at equal budget.	Computed: range occupancy. Prior: the 3.7× / 1.2× comparison.
Date sort	Dates in MM/DD/YYYY were sorted as strings, which orders them wrongly across year boundaries (“01/01/2021” before “12/31/2020”), scrambling the return series at every year end.	Dates parsed with an explicit format and sorted chronologically before returns are computed.	Prior: 5_Paper_Calculate_LogReturns.py, date-parse block.
Warnings silenced	<code>warnings.filterwarnings('ignore')</code> in the imports cell silenced third-party noise and, with it, the project’s own <code>UserWarning</code> guards.	A second filter re-enables <code>UserWarning</code> from the notebook’s own code; a regression check fails if it is removed.	Prior: notebook cell 0; <code>verify_notebook.py</code> check r17.
Discriminative AUC direction	Ranking ascending on raw AUC put an anti-predictive classifier (0.30) above an indistinguishable one (0.50).	Ranked on $ AUC - 0.5 $, with a z-score against the empirical null.	Prior: repository documentation.
GARCH fits at the integrated boundary	Maximum-likelihood GARCH fits converged, with no convergence warning, to persistence 1 and then simulated explosive paths. On one SHANGHAI walk-forward fold in an earlier production run, Wasserstein distance was 74.8, 7.8 and 43.0 across three seeds against a run-wide median of 0.0025. Whether a boundary fit explodes flipped between environments.	Two-stage fit: unconstrained persistence always recorded; refit under persistence $\leq 1 - 10^{-4}$ only at the boundary (Section ??). Library versions now recorded.	Computed: boundary counts and log-likelihood cost. Prior: the explosive-path Wasserstein values.

Table ??, continued

Defect	What was wrong, and its effect	Resolution	Evidence
Downstream utility not pooled	The per-seed file named <code>pooled_downstream_utility.csv</code> held one market's test set ($n = 486\text{--}501$), below the sample size at which QLIKE is stable, under a name promising the pooled set. The shuffled control beat a genuine generator in 33 of 75 control-versus-generator comparisons.	Per-market draws saved; one backtest per seed on all markets concatenated ($n = 2,479$). The metric is unchanged.	Computed: pooled n and results. Prior: the 33 of 75 count.
Fin-GAN name collision	The transposed-convolution WGAN-GP generator was called <code>FinGAN</code> , the name of a published and different model ^[41] that forecasts and classifies returns.	Renamed <code>CNN-WGAN-GP</code> throughout; the data layer reads result files under either name.	Prior: repository history.

B The stylised facts and the metrics that test them

Table 12. The ten stylised facts of financial returns (Cont^[7]; Mandelbrot^[9]; Ding, Granger & Engle^[8]) and the metric that tests each. Family names as in Table ??.

#	Stylized fact	Plain English	Mathematical statement	Our metric and family
1	Heavy tails	Extreme events occur far more often than a bell curve predicts.	$P(r > x) \sim x^{-\alpha}$, $\alpha \approx 2.5\text{--}3.2$ (BRICS Hill est.)	<code>tail_index_diff</code> (Hill 1975 ^[38]) · FIDELITY; <code>kurtosis_diff</code> · DESCRIPTIVE ($\mathbb{E} X ^4 = \infty$ for $\hat{\alpha} < 4$, so the sample statistic diverges with n — Section ??)
2	Near-zero autocorrelation	Knowing today's direction gives no useful information about tomorrow's.	$\text{Corr}(r_t, r_{t+k}) \approx 0$ for $k \geq 1$	<code>acf_returns_mae</code> · TEMPORAL
3	Volatility clustering	Large moves tend to be followed by large moves.	$\text{Corr}(r_t , r_{t+k}) > 0$ for $k = 1, \dots, 100+$	<code>acf_absolute_mae</code> , <code>acf_squared_mae</code> · TEMPORAL
4	Long memory in volatility	The clustering effect persists for hundreds of days.	$\text{ACF}(r_t) \sim k^{-\beta}$, $\beta \in (0, 1)$; $H > 0.5$	<code>hurst_diff</code> · TEMPORAL
5	Gain/loss asymmetry	Crashes are sharper and more extreme than equivalent-size rallies.	$\text{skewness}(r) < 0$ for equity indices	<code>skewness_diff</code> · DESCRIPTIVE ($\mathbb{E} X ^3 = \infty$ in 4/5 BRICS markets for $\hat{\alpha} < 3$). Modelled explicitly only by GJR-GARCH ^[40] , via its leverage term; none of the three GANs models it explicitly
6	ARCH effects	Return variance changes over time; it is not constant.	$\text{Var}(r_t \mid \mathcal{F}_{t-1}) = \sigma_t^2$ (time-varying)	<code>arch_pvalue_diff</code> , <code>arch_stat_diff</code> · DESCRIPTIVE

#	Stylized fact	Plain English	Mathematical statement	Our metric and family
7	Conditional heavy tails	After removing time-varying variance, residuals are still non-Gaussian.	$\varepsilon_t = r_t / \sigma_t$ has kurtosis > 0 (GARCH residuals)	resid_kurtosis_diff · TEMPORAL (Gaussian QMLE, to keep the test able to fail; circularity with the GARCH generators is discussed in Results)
8	Extreme events	Very large days occur more than 4.6 % of trading days.	$P(r > 2\sigma) > 4.6\%$ (Gaussian baseline)	extreme_events_diff · FIDELITY
9	Distributional match	The full shape of the return distribution must be reproduced.	$W_1(p_g, p_{\text{data}}) \approx 0$; $\hat{F}_{\text{syn}} \approx \hat{F}_{\text{real}}$	wasserstein, quantile_mse, energy_distance · FIDELITY
10	Indistinguishability	A classifier trained to separate real from synthetic should perform at chance level.	$\text{AUC}(\text{classifier}) \rightarrow 0.5$ at GAN optimum	discriminative_auc_dist, discriminative_auc_absz · TEMPORAL

C Code and design audit

Each entry records an issue found in the pipeline’s code or design, the evidence for it, and its status.

KS / Welch tests assumed i.i.d. — CRITICAL

FIXED

$n_{\text{eff}} = n / (1 + 2 \sum_k \rho_k) \ll n$ for financial series (ACF of $|r|$ sums to ≈ 5 – 15 for BRICS), so the KS statistic is inflated and its p -values anti-conservative. **Fix:** Wasserstein as the primary distributional metric, plus ARCH-LM, Hurst, energy distance and discriminative AUC.

Pointwise MSE/MAE had no semantic meaning

FIXED

MSE compared $r_{\text{real}}(t)$ with $r_{\text{syn}}(t)$, but the two series are independently generated and no temporal alignment exists between their indices. **Fix:** quantile MSE (compares sorted distributions, i.e. by rank rather than by index) and energy distance (Székely & Rizzo 2004^[11]).

Composite ranking: fidelity/temporal split + control exclusion

FIXED

The shuffled control won the unweighted composite, avg_rank 1.24 against 2.47, because permutation-invariant metrics dominated. **Fix:** metrics split into FIDELITY (7) and TEMPORAL (7); kurtosis_diff and skewness_diff demoted to DESCRIPTIVE (population moments undefined at Hill $\hat{\alpha}$ 2.54–3.21); composite_rank = mean of the two family ranks; control excluded from rank competition (NaN ranks) and appended as a reference line.

Discriminative AUC: direction inverted, and no null calibration

FIXED

Ascending rank on raw AUC put 0.30 above 0.50; the optimum is 0.5, not 0. The null is not exactly 0.5 either — measured at 0.506 ± 0.084 over 15 real-vs-real half-splits, so an observed 0.62 is consistent with a perfect generator. **Fix:** rank on $|AUC - 0.5|$, report a z -score against the empirical null, and do not shuffle the CV folds (shuffling moves the null to 0.584, because 20-day windows overlap by 19 observations).

Budgets specified in epochs rather than gradient steps

FIXED

An epoch is $\lfloor n_{\text{windows}} / \text{batch_size} \rfloor$ steps — data-dependent, and walk-forward folds differ in length by $5\times$. **Evidence:** this produced a $492\times$ asymmetry in generator updates in one run, when a smoke-test override

of epochs reached two models and not the third; the affected model read as an architectural failure until the budget was measured. **Fix:** all budgets in gradient steps, declared in one configuration cell, with `TimeGAN.joint_steps == QuantGAN.train_steps == CNN-WGAN-GP.train_steps` asserted at run time.

Both ARCH-LM variants demoted to descriptive; neither reliable across regimes REVISED

80-replication MC ($n = 1,200$, GARCH(1,1), Gaussian): `pvalue_diff` 99% accurate against 76% for `stat_diff`, whose null sd is ≈ 46.4 (two draws from the same process gave LM 56.8 and 124.8). On real BRICS data both real and shuffled p -values underflow to 0.0, so $|\Delta p| = 0.000000$ and the control is rated a perfect match. **Resolution:** both reported for diagnostic transparency, neither ranked; ACF-MAE is the primary volatility-clustering signal (88% MC accuracy, continuous, no saturation regime).

Min-max normalisation gave the generator an offset to learn before any dynamics FIXED

At Hill $\hat{\alpha}$ 2.54–3.21 one extreme day sets the whole scale. Min-maxing MOEX to $[-1, 1]$ leaves 98% of the data in 14.3% of the range with median +0.233. **Fix:** $z\text{-score} + \tanh(z/3)$, inverted by $z = 3 \operatorname{arctanh}(y)$. **Evidence:** changing only this moved QuantGAN's output sd from $3.7\times$ real to $1.2\times$ real at identical budget. Sigmoid to $[0, 1]$ was also rejected: it makes negative returns unrepresentable and $\operatorname{arctanh}$ explodes as $y \rightarrow 1$.

No outlier clipping on log returns — deliberate DECISION

A clip line existed in the preprocessing notebook, commented out, and was kept that way. Clipping at the 0.5th/99.5th percentiles removes exactly the observations that determine kurtosis, tail index and the extreme-events metric. Adams et al. (2019)^[6] show winsorising worsens distributional misfit.

Econometric models exempt from budget parity, and fitted on raw returns DECISION

Maximum-likelihood fitting has no gradient-step analogue, so the parity assertion is scoped to the gradient family and the econometric rows carry n/a rather than a fabricated update count. They are also fitted on raw log returns $\times 100$ rather than the project's $\tanh$ normalisation, which would compress the very variance dynamics GARCH exists to model; the $\times 100$ scaling is numerical, since `arch`'s optimiser converges poorly near 10^{-2} .

WGAN-GP gradient penalty is correct (QuantGAN & CNN-WGAN-GP) OK

$\hat{x} = \varepsilon x_{\text{real}} + (1 - \varepsilon)x_{\text{fake}}$; `requires_grad_(True)`; gradients via `torch.autograd.grad` with `create_graph=True`; `fake.detach()` prevents spurious accumulation. Implements Gulrajani et al. (2017)^[3] Eq. 3.

GARCH fits at the integrated boundary: two-stage fitting FIXED

12 of 180 econometric fits in this run reached persistence within 10^{-6} of 1 (NIFTY50 fold 0 GARCH; SHANGHAI main GARCH; SHANGHAI fold 3 GARCH; SHANGHAI fold 3 GJR-GARCH) and were refit under persistence $\leq 1 - 10^{-4}$; output was generated from the constrained parameters and both parameter sets are stored (Section ??). An earlier version raised and stopped the run on such fits, which would have kept SHANGHAI from completing on any seed; it was replaced because stopping the benchmark is not a treatment of the fit. The post-generation guard still raises outside $1/10$ – $10\times$ the training sd.

Downstream QLIKE: no degenerate cells OK

No model-seed cell returned a positive QLIKE in this run; median and mean are both reported regardless.

Downstream utility pooled across markets FIXED

One file, one backtest per model per seed, n of 2,479 over 5 markets. Before 2026-09-10 the pipeline

wrote one file per market-seed under this name, at that market's n (486–501) — below QLIKE's $n \gtrsim 600$ stability threshold, where the shuffled control's QLIKE beat a genuine generator's in 33 of 75 control-versus-generator comparisons (44%) — a prior measurement on those files.

Shuffled control outperforms genuine GANs on ordering-sensitive metrics

OPEN

On $ACF(r^2)$ MAE the control beats all 3 GANs, and on $ACF(r)$ MAE the models that clear it do so by 0.0003–0.0006. These metrics are *not* permutation-invariant — they separate the econometric models from the control cleanly — so the headline count stays at 9 of 19 and FIDELITY_COLS/TEMPORAL_COLS are unchanged. Recorded as a result about the generators, and as evidence that permutation-invariance in practice is a property of the comparison set rather than of a metric alone.

Earlier attribution of TimeGAN's collapse to tanh saturation

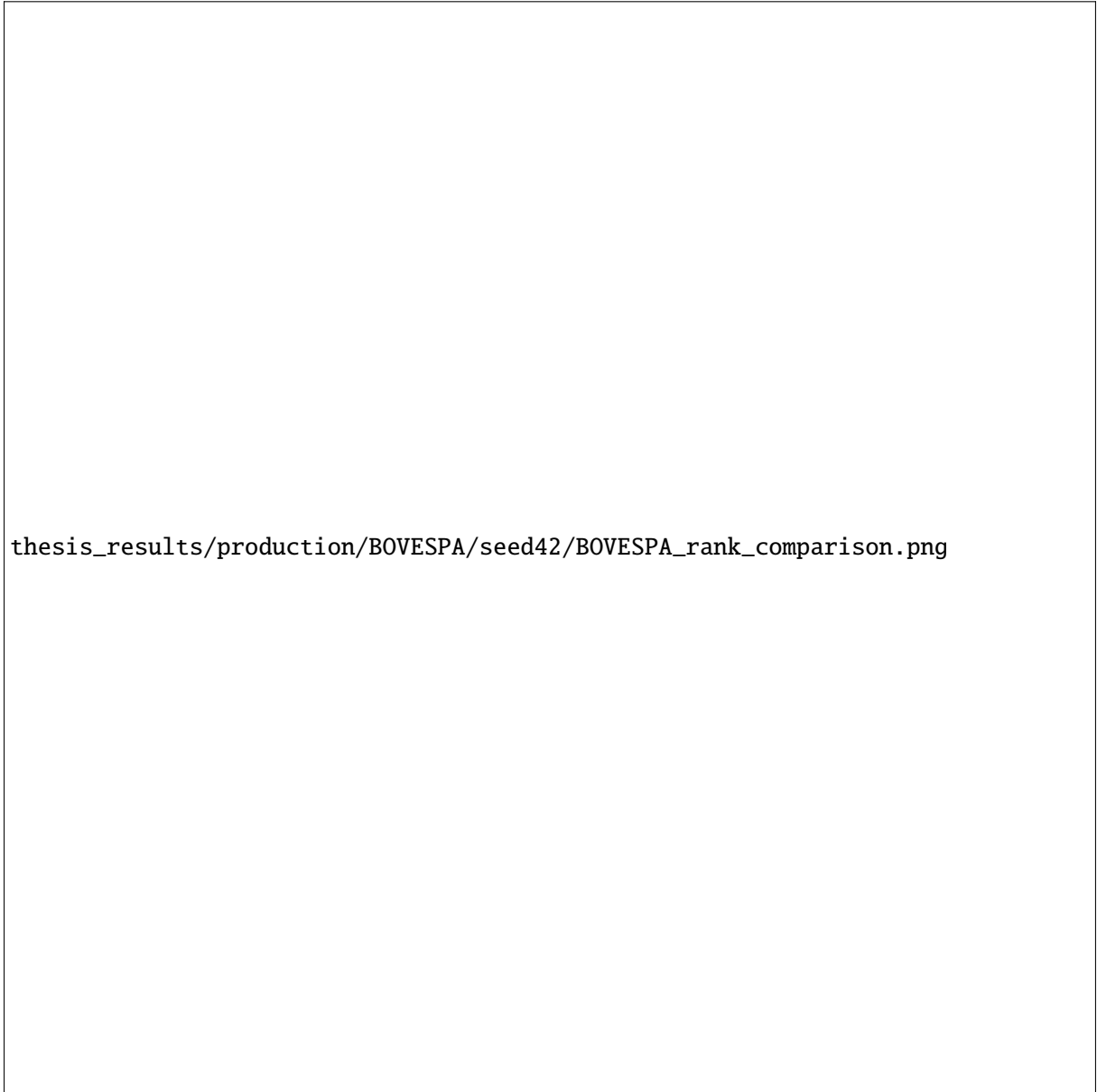
WITHDRAWN

A previous draft explained TimeGAN's degenerate output as tanh saturation in the Recovery network. This run's [DIAG] latent instrumentation does not support that: saturation never exceeds 3.1% over 90 readings against the 50% threshold the diagnostic exists to detect, and reconstruction sd never differs from target by more than 0.0014, so the autoencoder is functioning. The collapse is real — the guard fired 18 times, all TimeGAN (16), QuantGAN (2), mean $|ACF(1)| = 0.533$ — but the mechanism is open. The earlier explanation is withdrawn rather than quietly dropped.

D Metric diagnostics

thesis_results/production/BOVESPA/seed42/BOVESPA_metrics_heatmap.png

Figure 9. Metric values for BOVESPA, seed 42, drawn by the pipeline. Each cell is one metric for one model.



thesis_results/production/BOVESPA/seed42/BOVESPA_rank_comparison.png

Figure 10. Rank comparison for BOVESPA, seed 42, drawn by the pipeline. It includes the unweighted mean over all metrics, `avg_rank`, which is kept for comparability only and is never the selection criterion; models are selected on the composite rank (Section ??).

E Data and code availability

Data. Daily closing prices for the five indices were obtained from a public market-data provider and are not redistributed. The processed log-return series are stored as CSV files under `data/processed_files/`, and their 80/10/10 splits as Parquet files in its `train`, `valid` and `test` subdirectories; the processed series are reproduced from the source price files by `5_Paper_Calculate_LogReturns.py`.

Code. The pipeline — all five generators, every metric and the ranking — is `3_4_integrated_pipeline.ipynb`. This report is built by `generate_report.py` from the run artifacts in the results directory through `report_data.py`. `verify_notebook.py` is the regression check. The package requirements file sets lower bounds rather than pinning exact versions (for example `arch>=7.0`); the exact versions used by this run are those in the provenance block. The seeds are 42, 43, 44; walk-forward fold boundaries are deterministic; run-to-run determinism on the GPU is deliberately not enforced (Section ??).